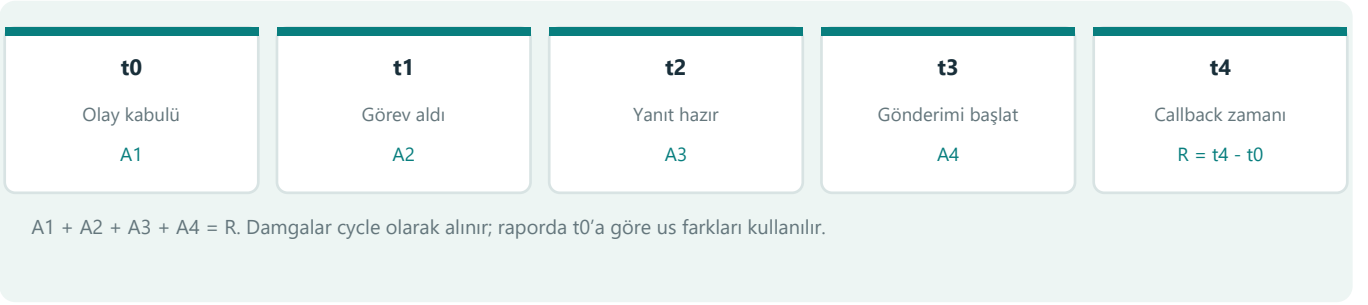


Butondan UART'a.

Uygulama kodunu
adım adım okumak



Kesmeden göreve, queue'dan DMA'ya, zaman damgasından ölçüm kaydına. Mevcut kodun amacı, sırası ve bağlantıları; ilk kez öğrenen biri için.

İNCELEME KAPSAMI	KAYNAKTAKİ SEÇİM
22 bölüm / 59 uygulama fonksiyonu	STM32F746 / CMSIS-RTOS2
Kaynak: firmware/Button_Task/Core	LARGE_FIFO / 256 öge / 921600 baud
26 Eylül 2026 / Git HEAD ffa7e06	Kod açıklaması; yeni kart ölçümü değil

HTML sürümünde bölüm araması, kaynak kutuları ve adım adım akış gezgini bulunur. PDF, açıklama içeriğinin tamamını taşır.

Okuma haritası

İlk okumada bölüm sırasını izleyin. Sonraki okumalarda bu içindekiler ve PDF yer imleriyle doğrudan ilgili akışa geçebilirsiniz.

01 Bu programı neden yazıyoruz?	3
02 Kodu okumak için gereken küçük sözlük	5
03 Büyük resim: queue'lar neden var?	7
04 Sabitleri ve veri sözleşmelerini kurmak	9
05 Zamanı ölçmek: app_timestamp	11
06 Açılış: kaynaklar hangi sırayla kuruluyor?	13
07 PC'den gelen bir bayt nasıl komuta dönüşür?	15
08 Komutu doğrulamak: app_protocol ayrıştırıcısı	17
09 Deneyin kumandası: CFG, START, STOP, DUMP	19
10 Buton kenarı: app_button.c	21
11 ButtonTask: olayı yanıt mesajına çevirmek	23
12 TelemetryTask: periyot ve yapay CPU yükü	25
13 UartTxTask: UART'ın tek sahibi	27
14 DMA gönderimi ve tamamlanma callback'leri	29
15 Ring buffer: bir olayın parça parça tamamlanan kaydı	32
16 Protokol: kaydı 64 bayta dönüştürmek	36
17 Deney sonu: STOP'tan DUMP_END'e	39
18 Kodda mevcut üç TX modu	41
19 Bir olayı baştan sona birlikte izleyelim	43
20 Yalnızca bu projede kullanılan RTOS çağrıları	45
21 Dosyalar, bağımlılıklar ve fonksiyon bulma kılavuzu	47
22 Kaynak kapsamı ve okuma notları	51

BÖLÜM 01 / 22

01 Bu programı neden yazıyoruz?

Bir butona basıldığında bilgisayara `BTN, . . . , PRESSED` mesajı göndermek istiyoruz. Fakat bu projenin asıl sorusu yalnızca mesajın gidip gitmediği değil: **İşlemci başka işlerle uğraşırken, kabul edilen buton olayı için yanıt kaç mikrosaniyede tamamlanıyor ve zaman nerede harcanıyor?**

Bunun için uygulama aynı işlemciye üç iş veriyor. `ButtonTask` buton yanıtını oluşturuyor. `TelemetryTask` düzenli mesaj üretiyor ve ayarlanabilen bir hesap döngüsüyle CPU'yu meşgul ediyor. `UartTxTask` hazırlanmış mesajları sırayla UART üzerinden gönderiyor. Her butonun yolculuğu beş zaman damgasıyla RAM'e kaydediliyor. Deney durdurulduktan sonra ayrıntılı kayıtlar bilgisayara aktarılıyor.

Bu düzen, yükün etkisini görünür kılmak için kurulmuş. `TelemetryTask`'ın yüksek önceliği CPU beklemesini; ortak TX kuyruğu ise gönderim sırası beklemesini oluşturabiliyor. Bunlar bu uygulamanın ölçtüğü davranışlar.

Bu rehberin dayanağı

İncelenen kaynak kökü `firmware/Button_Task/Core`. Rehber, 26 Eylül 2026 tarihinde çalışma klasöründeki kaynak dosyalar üzerinden hazırlandı. Git HEAD: `ffa7e06`. Buradaki açıklamalar kaynak kodun davranışını anlatır; kartta yeni bir deney çalıştırıldığı veya mevcut ELF'in bu kaynakla aynı olduğu iddiasını taşımaz.

İnsan/AI tarafından yazılan uygulama katmanı olan `app_*.c` ve `app_*.h` dosyalarının tamamı kapsamdadır. `main.c` içindeki uygulamayı başlatan bağlantı ve UART hızı seçimi de anlatılır. HAL sürücülerinin, FreeRTOS çekirdeğinin ve CubeMX'in çevre birimi kurulum kodlarının iç algoritmaları bu rehberin konusu değildir. Uygulama bunları çağırdığında yalnızca o çağırının buradaki görevi açıklanır.

"Sıfırdan yazarken ne düşünülmüş?" anlatımı, dosyaların bağımlılıklarından ve kaynak yorumlarından kurulan bir öğrenme sırasıdır. Yazarın gerçek yazım geçmişini veya zihninden geçenleri bildiğimiz anlamına gelmez. Her bölümde önce çözülmek istenen ihtiyacı, sonra onu gerçekleştiren satırları takip edeceğiz.

Önce mevcut sürümü doğru sabitleyelim

Konu	Bu kaynakta gerçek değer / yer	Okurken anlamı
Varsayılan TX modu	<code>app_config.h:34-38: LARGE_FIFO</code>	Harici bir mod tanımı yoksa bu dal derlenir.
Ortak TX kuyruğu	<code>app_config.h:93: 256 öge</code>	Aynı dosyadaki 64 açıklaması güncel değeri göstermiyor.
UART hızı	<code>main.c:221,238: 921600 baud</code>	README'deki 115200 bu kaynağın seçimi değil.
Uygulama başlatma	<code>main.c:139: App_RTOS_Init</code>	Bazı başlık yorumlarındaki <code>freertos.c</code> adresi geçerli değil.
Uygulama görevleri	<code>UartTxTask, ButtonTask, TelemetryTask</code>	Bunlara ek olarak CubeMX <code>defaultTask</code> 'ı da oluşturuluyor.
Kaynak klasörü	<code>firmware/Button_Task</code>	Eski belgelerdeki <code>Cubeide_Manuel</code> yolu yerine bu klasör okunmalı.

.cproject ve mevcut Debug derleme kurallarında TX modunu değiştiren bir `-DAPP_TX_MODE_...` tanımı görülmüyor. Bu yüzden ana anlatım 256 öğeli ortak FIFO dalına göre ilerliyor. Kodda bulunan ORIGINAL ve DUAL_QUEUE dalları ayrıca açıklanıyor; bunlar CFG komutuyla değişen çalışma zamanı ayarları değil, derleme zamanı seçimleri.

İlk okumada izleyeceğimiz rota

Adım	Okunacak yer	Burada cevaplanan soru
1	app_config.h, app_rtos.h	Sabitler ve taşınan veri nedir?
2	main.c, app_rtos.c	Kaynaklar hangi sırayla hazırlanıyor?
3	app_cmd_rx.c, komut ayrıştırıcı, kontrol fonksiyonu	Deney nasıl başlıyor?
4	app_button.c, ButtonTask_Run	Bir basış nasıl yanıt mesajına dönüşüyor?
5	TelemetryTask_Run	Bu sırada yük nasıl üretiliyor?
6	UartTxTask_Run, uart_tx_send, callback'ler	Mesaj nasıl gönderiliyor ve bitiş nasıl anlaşılıyor?
7	app_ring_buffer.c, protokol oluşturucuları	Ölçüm nasıl saklanıyor ve dışarı çıkarılıyor?

Dosya sırası ile yürütme sırası aynı değildir. Örneğin `app_tasks.c` başında `uart_tx_send` yazılıdır ama bu, kart açılır açılmaz ilk o fonksiyonun çalışacağı anlamına gelmez. Normal fonksiyonlar çağrıldığında; görev girişleri scheduler seçtiğinde; callback'ler ilgili donanım olayı işlendiğinde çalışır.

BÖLÜM 02 / 22

02 Kodu okumak için gereken küçük sözlük

Görev, kesme ve bekleme

Görev (task/thread), RTOS'un çalıştırdığı bir fonksiyondur. Bu projedeki üç görev `for ( ; ; )` içinde yaşar. Sonsuz döngü olması sürekli CPU tükettiği anlamına gelmez: kuyruktan veri beklerken, semaphore beklerken veya `osDelayUntil` içindeyken görev bloklanır. RTOS o sırada hazır olan başka bir görevi çalıştırabilir.

Kesme (interrupt), donanım olayına verilen kısa tepkidir. Buton kenarı ve UART olayları uygulama callback'lerine bu bağlamda ulaşır. Kesmede normal görev gibi uzun süre beklenmez. Bu nedenle `ButtonQueue` ve `CmdQueue`'ya kesmeden yapılan `Put` çağrılarının son parametresi sıfırdır.

Hazır olmak çalışıyor olmak değildir. `ButtonQueue`'ya olay gelince `ButtonTask` hazır olabilir; yüksek öncelikli `TelemetryTask` hâlâ çalışıyorsa `ButtonTask` onun CPU'yu bırakmasını bekler. Bu aralık ölçümde görünür.

Bu C ifadeleri ne söylüyor?

İfade	Bu projede nasıl okunmalı?
<code>uint8_t / uint16_t / uint32_t</code>	8 / 16 / 32 bit işaretli sayı. Bayt, olay kimliği ve zaman sayacı için kullanılıyor.
<code>bool</code>	true/false bilgisi; örneğin deney çalışıyor mu?
<code>struct / typedef</code>	Birbiriyle ilgili alanları tek paket yapar ve o pakete isim verir.
<code>enum</code>	Bir durumun anlaşılır adlarını tanımlar: <code>TX_OK</code> , <code>CMD_STOP</code> gibi.
<code>&evt</code>	evt nesnesinin adresi. Kuyruk, bu adresteki içeriği kopyalayabilir.
<code>*t3 = ...</code>	Adresi verilen değişkenin içeriğini değiştirir; çağırana sonuç döndürmenin bir yolu.
<code>item.frame / rec->t0</code>	Nesnenin alanı / işaretçinin gösterdiği nesnenin alanı.
<code>static</code> , dosya düzeyinde	İsim yalnızca o .c dosyasının içinde görünür. Depolama program boyunca yaşar.
<code>static</code> , fonksiyon içinde	Değer çağrılar arasında korunur; initialized buna örnektir.
<code>extern</code>	Değişkenin gerçek tanımı başka dosyada. Yeni kopya oluşturma, mevcut olana eriş.
<code>volatile</code>	Değeri kesme gibi dış bir akış değiştirebilir; derleyici erişimi yok saymasın. Kilitleme sağlamaz.
<code>const</code>	Bu erişim üzerinden içeriği değiştirme. Fonksiyon sözleşmesini açık eder.
<code>0u / 1u</code>	İşaretsiz sayı sabiti; sayaçlarla yapılan işlemlerde kullanılır.
<code>(void)çağrı(...)</code>	Fonksiyon çalışır, dönüş değeri bilinçli olarak kullanılmaz.
<code>#ifdef / #if</code>	Derlemeden önce hangi kodun programa gireceğini seçer. RTOS görevi değildir.

Başlık dosyası `.h`, diğer dosyaların bilmesi gereken tipleri, sabitleri ve fonksiyon bildirimlerini verir. `.c` ise bu fonksiyonların gövdelerini ve modülün özel durumunu içerir. `#ifndef ... #define ... #endif` başlık koruması, aynı başlığın tekrar işlenmesini engeller. `#include` bir fonksiyonu çalıştırmaz.

`app_hw.h` içindeki `extern UART_HandleTypeDef APP_UART_HANDLE;` satırında makro `huart1` adına açılır. `huart1`'in gerçek tanımı `main.c`'dedir. Böylece gönderici ve alıcı uygulama modülleri aynı UART nesnesini kullanır.

Dört farklı "kap" birbirine karıştırılmamalı

Yapı	İçinde ne tutuluyor?	Ne için?
RTOS queue	Kopyalanmış olay / mesaj / komut öğeleri	Bir işin başka yürütme akışına teslimi
Ring buffer	Olayın t0..t4 kaydı ve sonucu	Deney geçmişini RAM'de tutmak
Semaphore	En fazla bir bildirim jetonu	UART tamamlandı veya DMA hatası oldu diye uyandırmak
Event/thread flag	Bir veya birkaç durum biti	Deney açık veya yeni iş var bilgisini duyurmak

Bu ayırım bütün tasarımın anahtarıdır: **iş teslimi, sonuç arşivi ve uyandırma aynı ihtiyaç değildir.**

BÖLÜM 03 / 22

03 Büyük resim: queue'lar neden var?



Buton kesmesi, ButtonTask'ın fonksiyonunu doğrudan çağırıyor. Küçük bir ButtonEvent_t oluşturup ButtonQueue'ya kopyalıyor. Kesme işini bitiriyor; RTOS uygun olduğunda ButtonTask bu olayı alıyor. Böylece olayın kimliği ve zamanı, görev hemen çalışmasa da korunuyor.

ButtonTask ve TelemetryTask aynı fiziksel UART'a mesaj üretir. İkisi de UART gönderimini kendi başına başlatmaz; TxQueue'ya hazır çerçeve bırakır. UartTxTask bu kuyruğun tek tüketicisidir. Böylece hangi çerçevenin ne zaman gönderileceği tek bir noktadan yürür ve DMA'nın kullanacağı arabellek tek yerde yönetilir.

CmdQueue üçüncü bir ihtiyaç içindir. UART alıcı kesmesi komutun baytlarını toplar; tamamlanan ham satırı kuyruğa bırakır. CRC kontrolü, sayıya çevirme ve START/STOP kararı UartTxTask bağlamında yapılır. Komut almak için dördüncü bir uygulama görevi oluşturulmaz.

Kim üretir, kim tüketir?

Nesne	Üreten	Tüketen	Taşınan öğe / kapasite
ButtonQueue	Button_HandleEXTI	ButtonTask_Run	ButtonEvent_t / 8
TxQueue	ButtonTask ve TelemetryTask	UartTxTask_Run	TxItem_t / mevcut modda 256
CmdQueue	CmdRx_HandleByteReceived	UartTxTask_Run	CmdLine_t / 4
TxDoneSem	TX tamamlanma veya DMA hata callback'i	uart_tx_send	Veri paketi değil, bildirim / en fazla 1
RunFlags.RUN_BIT	control_handle_command	TelemetryTask_Run	Deneyin çalışma izni

FIFO, önce eklenen öğenin önce alınmasıdır. Ortak TX kuyruğunda TEL1, TEL2, BTN1 sırası oluşmuşsa ButtonTask'ın önceliği, BTN1'i kendiliğinden kuyruğun başına geçirmez. **Görev önceliği CPU'yu kimin alacağını; FIFO sırası hangi mesajın sıradaki olduğunu belirler.** Bu projedeki Put çağrılarında mesaj önceliği alanı 0'dır; paketle gelen CMSIS-FreeRTOS uyarlaması da bu alanı kullanmaz.

Queue veriyi kopyalar

osMessageQueueNew(..., sizeof(TxItem_t), ...), her öğenin kaç bayt olduğunu kaydeder. Daha sonra osMessageQueuePut(..., &item, ...) çağrısı başarılı olduğunda item'ın o anki içeriği kuyruğa kopyalanmıştır. Kuyrukta yalnızca yerel değişkenin adresi saklanmaz. Bu nedenle ButtonTask sonraki döngüde başka bir yerel item oluştursa da önceki mesajın kuyruğa alınmış kopyası vardır.

`osMessageQueueGet(..., &item, ...)` ise öğeyi kuyruktan çıkarır ve alıcının item değişkenine kopyalar. Burada sahiplik el değiştirir: hazırlanmış mesaj artık `UartTxTask`'ın elindedir. Daha sonra DMA için ayrıca kalıcı `s_tx_buffer`'a kopyalanacaktır; o kopyanın nedeni DMA'nın fonksiyon çağrısından sonra da belleği kullanmasıdır.

Üç görev hangi anlarda çalışabilir?

Görev	Öncelik	Normal bekleme noktası
TelemetryTask	AboveNormal	RUN_BIT veya sonraki periyot; ortak FIFO doluysa Put beklemesi
ButtonTask	Normal	ButtonQueue; ortak FIFO doluysa Put beklemesi
UartTxTask	Low	TxQueue'da veri veya TX tamamlanma semaphore'u

Yüksek öncelikli görev bloklanınca daha düşük öncelikli görev çalışabilir. Örneğin `TelemetryTask` dolu `TxQueue`'da beklediğinde CPU'yu bırakır; `UartTxTask` kuyruktan mesaj çekip yer açabilir. Kuyruk beklemesini aktif hesap döngüsüyle karıştırmayın: hesap döngüsü CPU tüketir, bloklı bekleme tüketmez.

`defaultTask` da Normal öncelikte oluşturulur ve sürekli `osDelay(1)` yapar. Uygulama mesajı üretmez. Dolayısıyla "üç uygulama görevi" ifadesi doğrudur; sistemde yalnızca üç RTOS görevi olduğu anlamına gelmez. Çekirdeğin kendi görevleri de bu sayıya dahil değildir.

BÖLÜM 04 / 22

04 Sabitleri ve veri sözleşmelerini kurmak

Sıfırdan kurarken önce modüllerin üzerinde anlaşacağı değerleri tanımlamak gerekir: çerçeve kaç bayt, kuyrukta ne taşınacak, bir olay nasıl tanınacak? `app_config.h` ve `app_rtos.h` bu anlaşmayı sağlar.

KAYNAK / `Inc/app_config.h` / satır 34-44

```
34 #if !defined(APP_TX_MODE_ORIGINAL) && \
35     !defined(APP_TX_MODE_LARGE_FIFO) && \
36     !defined(APP_TX_MODE_DUAL_QUEUE)
37 #define APP_TX_MODE_LARGE_FIFO
38 #endif
39
40 #if (defined(APP_TX_MODE_ORIGINAL) + \
41     defined(APP_TX_MODE_LARGE_FIFO) + \
42     defined(APP_TX_MODE_DUAL_QUEUE)) != 1
43 #error "TX modu için tam olarak BİR #define seçilmelidir."
44 #endif
```

İlk koşul, üç modun hiçbiri dışarıdan tanımlanmamışsa `LARGE_FIFO`'yu seçer. İkinci koşul, tanımlı modların sayısını toplar ve sonuç tam bir değilse derlemeyi durdurur. Böylece birbiriyle çelişen kuyruk düzenleri aynı programda etkin olmaz. Üstteki yorumun `ORIGINAL` demesine rağmen yürütülecek seçimi belirleyen satır 37'dir.

Zaman ve boyut sabitleri

Sabit	Değer	Kullanıldığı karar
<code>APP_FRAME_SIZE / PAYLOAD_SIZE</code>	64 / 62 bayt	Gönderilen toplam boyut / ASCII ve boşluk alanı
<code>CRC_INDEX / LF_INDEX</code>	62 / 63	CRC ve satır sonunun konumu; diziler sıfırdan başlar
<code>APP_CMD_LINE_MAX_LEN</code>	64	PC komutunda LF hariç en çok kaç bayt biriktirilebilir
<code>APP_DEADLINE_US</code>	20000 us	Tamamlanan buton yanıtının OK/LATE sınıflaması
<code>APP_DEBOUNCE_MS</code>	30 ms	Önceki kenarla bu kenar arasındaki sessizlik koşulu
<code>APP_TX_TC_TIMEOUT_MS</code>	100	Semaphore bekleme üst sınırı; 1 kHz tick ile 100 ms
<code>APP_TX_QUEUE_SEND_TIMEOUT_MS</code>	200	Ortak FIFO doluyorsa üreticinin en çok bekleyeceği tick
<code>APP_MIN_TELEMETRY_PERIOD_MS</code>	10	CFG'de daha küçük periyot 10'a çekilir
<code>APP_RING_BUFFER_CAPACITY</code>	256	RAM'de tutulacak en yeni olay kaydı sayısı
Varsayılan senaryo	200 ms, 0 iterasyon, kimlik 0	CFG gelmeden START verilirse kullanılacak ayarlar

20 ms deadline, 100 ms TX bekleme sınırı ve 200 ms kuyruk bekleme sınırı farklı şeylerdir. Deadline "bu olayı zamanında tamamladık mı?" sorusuna cevap verir. Diğerleri bir RTOS çağrısının ne kadar bekleyeceğini sınırlar.

Deadline aşılmıyorsa gönderimi otomatik iptal eden bir zamanlayıcı yoktur. Gönderim tamamlanırsa kayıt LATE olabilir.

Bu API'lere verilen beklemler tick cinsindedir. Projenin FreeRTOSConfig.h dosyasında configTICK_RATE_HZ 1000 olduğundan bir tick bir milisaniedir. APP_KERNEL_TICK_HZ bu varsayımı belgeler; çekirdeği tek başına o makro yapılandırmaz.

Olay paketi: ButtonEvent_t

KAYNAK / Inc/app_rtos.h / satır 20-35

```
20 /** ButtonQueue elemanı: ISR -> ButtonTask */
21 typedef struct {
22     uint32_t t0;           /* DWT cycle, ISR girişinde alınan */
23     uint32_t t0_abs_ms;    /* START'tan itibaren ms (kronoloji için) */
24     uint16_t event_id;
25     uint16_t ring_index;
26     uint8_t  scenario_id;
27 } ButtonEvent_t;
28
29 /** TxQueue elemanı: ButtonTask/TelemetryTask -> UartTxTask */
30 typedef struct {
31     uint8_t type;          /* MSG_TYPE * */
32     uint16_t event_id;     /* BTN için: kaydı kapatırken doğrulanır */
33     uint16_t ring_index;   /* TEL için RING_INDEX_NONE */
34     uint8_t  frame[APP_FRAME_SIZE];
35 } TxItem_t;
```

ButtonEvent_t'deki t0 hassas cycle sayacıdır; t0_abs_ms ise deneyin kaçınıcı milisaniyesinde basıldığını gösterir. event_id basışın kimliğidir. ring_index bu olayın RAM kayıt dizisindeki yeridir. scenario_id, olayın hangi deney etiketiyle üretildiğini taşır. Bunlar mesaj henüz oluşmadan gereken bilgilerdir.

TxItem_t ise hazır frame[64] ile onu ölçüm kaydına bağlayan event_id/ring_index alanlarını taşır. type alanına BTN veya TEL yazılır; ancak mevcut send_item kararını type üzerinden değil, ring_index'in RING_INDEX_NONE olup olmaması üzerinden verir. Bu ayrımı, yapıda bir alan görmenin onun her yerde kullanıldığı anlamına gelmediğini gösterir.

CmdLine_t içinde len ve text[64] vardır. Alıcı ISR bu metnin sonuna NUL eklemek zorunda değildir; ayrıştırıcı len kullanarak kendi 65 baytlık tamponuna kopyalayıp NUL ekler. ScenarioConfig_t içindeki period_ms, load_iter, scenario_id ise o anki deney ayarlarıdır. scenario_id yalnızca etikettir; S0/S1 gibi isimlerin periyot/yük tablosunu firmware içinde seçmez. Ayarlar CFG ile ayrı ayrı gelir.

Paylaşılan durum nerede yaşar?

Değişken	Yazan	Okuyan / amacı
g_cfg	Başlangıç tanımı ve CFG işleyicisi	Telemetri ayarları, butonun senaryo etiketi, dump
g_running	START / STOP işleyicisi	Buton kabulü ve telemetri döngüsü
g_run_start_ms	START işleyicisi	Deney başlangıcına göre ms hesabı
g_dwt_ok	App_RTOS_Init	BOOT mesajında sayaç kontrol sonucu
g_tx_done_cycles	TX tamamlanma callback'i	uart_tx_send için t4
g_tx_dma_error	Gönderim başlangıcı ve DMA hata callback'i	Uyanmanın başarı mı hata mı olduğunu ayırmak

Bu değişkenlerin tanımları app_rtos.c'de, extern bildirimleri app_rtos.h'dedir. Aynı isimler her dosyada yeni bir durum yaratmaz. ISR ile paylaşılan ilgili alanlardaki volatile, okuma/yazmanın görünür kalmasını sağlar; çok adımlı işlemleri kendiliğinden bölünmez yapmaz. Ring buffer güncellemelerinde bu yüzden ayrıca kritik bölge vardır.

BÖLÜM 05 / 22

05 Zamanı ölçmek: app_timestamp

Bu deneyde birkaç mikrosaniyelik farkları görmek istiyoruz. Görev uyutmak için kullanılan 1 ms tick ile hassas ölçümün işi farklıdır. Uygulama hassas zaman noktaları için işlemcinin DWT cycle sayacını okur. Bu modül HAL kütüphanesi değildir; uygulamanın kendi ölçüm yardımcı kodudur.

KAYNAK / Src/app_timestamp.c / satır 3-18

```

3  #define DWT_LAR_UNLOCK_KEY 0xC5ACCE55u
4
5  bool Timestamp_Init(void)
6  {
7      CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
8      /* Cortex-M7'de DWT registerlari Lock Access Register ile kilitlidir;
9       * debugger bagli degilken acilmazsa CYCCNT hic saymaz. */
10     DWT->LAR = DWT_LAR_UNLOCK_KEY;
11     DWT->CYCCNT = 0u;
12     DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
13
14     uint32_t before = DWT->CYCCNT;
15     for (volatile int i = 0; i < 16; i++) {
16     }
17     return DWT->CYCCNT != before;
18 }
```

Timestamp_Init önce izleme altyapısının izin bitini açar. DWT'nin kilit erişim yazmacına anahtarı yazar; sayacı sıfırlar ve sayma bitini açar. Ardından before değerini alıp kısa bir volatile döngü geçirir. Son değer farklıysa sayaç ilerliyor demektir ve true döner. Bu test kesin bir frekans kalibrasyonu değil, sayacın hareket ettiğinin kontrolüdür.

Timestamp_Now() app_timestamp.h içinde static inline tanımlıdır ve yalnızca DWT->CYCCNT döndürür. Çağrıldığı noktada cycle sayacının fotoğrafını alır. Bu değeri tek başına "mikrosaniye" diye okumayız.

KAYNAK / Src/app_timestamp.c / satır 20-28

```

20  uint32_t Timestamp_DeltaToUs(uint32_t from, uint32_t to)
21  {
22     uint32_t delta_cycles = to - from;
23     uint32_t cycles_per_us = SystemCoreClock / 1000000u;
24     if (cycles_per_us == 0u) {
25         cycles_per_us = 1u;
26     }
27     return delta_cycles / cycles_per_us;
28 }
```

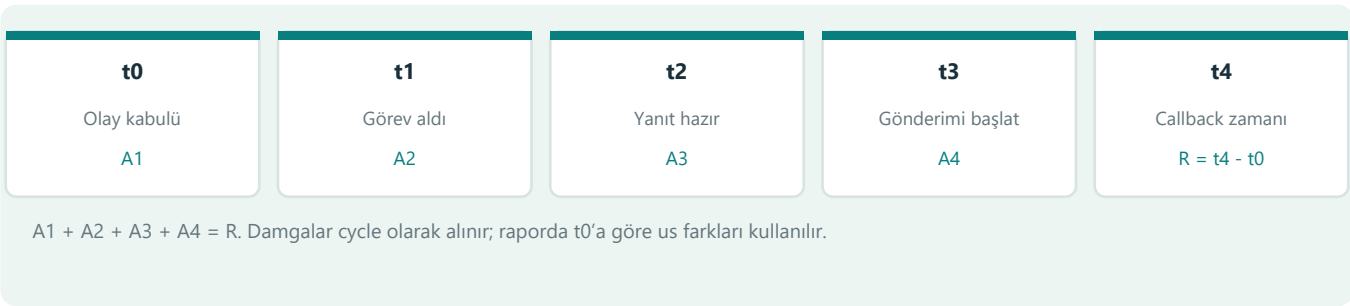
Timestamp_DeltaToUs(from, to), önce işaretli cycle farkını hesaplar. SystemCoreClock 216000000 ise bir mikrosaniyede 216 cycle vardır. Örneğin 21600 cycle farkı 100 us olur. Bölme tamsayı bölmesidir; mikrosaniyenin kesirli kısmı atılır. Saatten hesaplanan bölen sıfırsa kod sıfıra bölmeyi önlemek için 1 kullanır.

32 bit sayaç 216 MHz'de yaklaşık 19,88 saniyede başa döner. İşaretsiz çıkarma, **gerçek süre bir sayaç çevriminden kısa olduğu sürece**, sayaç arada sıfırdan geçmiş olsa da farkı doğru verir. Örneğin from=4294967280 ve to=16 ise fark 32 cycle'dır. Bu, sınırsız uzun süreleri ölçebileceği anlamına gelmez.

Beş damganın tam yeri

Damga	Gerçekte alındığı yer	Ölçüm sınırı
t0	Button_HandleEXTI'nin ilk işlemi	Uygulama buton işleyicisinin başlangıcı; kabul edilmeyen kenarda kullanılmaz
t1	ButtonQueue Get başarıyla döndükten hemen sonra	ButtonTask olayı işlemeye başladı
t2	BTN çerçevesi bittikten sonra, TX Put öncesi	Yanıt hazır, gönderim yoluna teslim edilecek
t3	uart_tx_send içinde DMA başlatma çağrısından hemen önce	UART gönderimini başlatma girişimi
t4	HAL_UART_TxCpltCallback içinde	UART tamamlanma olayı kesmede işlenirken

t0 fiziksel anahtar kontağının değiştiği anın donanım kaydı değildir. EXTI kesme girişinden ve HAL dağıtımından sonra Button_HandleEXTI'ye gelindiğinde alınır. t3 de ilk bitin telden çıktığı an değildir. t4 son bit tamamlandıktan sonra callback'in işlendiği andır. Dolayısıyla sınırlar yazılımsal ölçüm noktalarıdır; t4'te görevin sonradan uyanma gecikmesi dışarıda kalır, kesmenin işlenme gecikmesi kalabilir.



Aşama	Hesap	İçinde neler bulunabilir?
A1: olayı teslim alma	$t1 - t0$	ISR'daki kalan işler, kuyruk teslimi, ButtonTask'ın CPU beklemesi
A2: yanıtı hazırlama	$t2 - t1$	Alanları doldurma, BTN metni, CRC; araya giren daha öncelikli çalışma
A3: gönderime ulaşma	$t3 - t2$	Dolu kuyruğa yer bekleme, FIFO bekleme, görev zamanlaması, TX ön hazırlığı
A4: gönderimi tamamlama	$t4 - t3$	Başlatma maliyeti, fiziksel aktarım, tamamlanma kesmesinin işlenmesi
R: toplam yanıt	$t4 - t0$	Kabul edilmiş olaydan tamamlanma callback'ine kadar geçen zaman

Depodaki belgelerde bu aşamalar S1-S4 olarak geçiyor. Senaryo etiketleri S0-S5 ile karışmaması için rehberde A1-A4 adlarını kullanıyoruz; formüller aynı. Bu aralıklar yalnızca ilgili fonksiyonun saf CPU tüketimi değil, iki damga arasındaki geçen zamandır.

BÖLÜM 06 / 22

06 Açılış: kaynaklar hangi sırayla kuruluyor?

Başlangıçta g_running false'tur. Kart açılınca uygulama hemen deney üretmeye başlamaz. Önce çevre birimleri, RTOS nesneleri ve görevler hazırlanır; deney daha sonra START komutuyla açılır.

main.c'de önce sistem saati ve GPIO/DMA/UART başlatılır. Ardından osKernelInitialize gelir. defaultTask oluşturulur; USER CODE RTOS_THREADS bloğundaki App_RTOS_Init uygulamanın nesnelerini oluşturur. Son olarak osKernelStart scheduler'ı çalıştırır. Bu düzenin anlamı: görevler kuyruklara erişmeye başlamadan kuyrukların ve başlangıç durumunun hazır olmasıdır.

KAYNAK / Src/main.c / satır 134-147

```
134  /* Create the thread(s) */
135  /* creation of defaultTask */
136  defaultTaskHandle = osThreadNew(StartDefaultTask, NULL, &defaultTask_attributes);
137
138  /* USER CODE BEGIN RTOS_THREADS */
139  App_RTOS_Init(); /* kuyruklar, semaphore, event flag + UartTxTask/ButtonTask/TelemetryTask */
140  /* USER CODE END RTOS_THREADS */
141
142  /* USER CODE BEGIN RTOS_EVENTS */
143  /* add events, ... */
144  /* USER CODE END RTOS_EVENTS */
145
146  /* Start scheduler */
147  osKernelStart();
```

Scheduler başladıktan sonra main'in aşağıdaki boş while(1) döngüsü uygulamanın iş yaptığı yer değildir. Uygulamanın sürekli akışı üç görev fonksiyonuna dağılmıştır.

App_RTOS_Init adım adım

KAYNAK / Src/app_rtos.c / satır 43-59

```
43 void App_RTOS_Init(void)
44 {
45     static bool initialized = false;
46     if (initialized) {
47         return;
48     }
49     initialized = true;
50
51     g_dwt_ok = Timestamp_Init();
52     RingBuffer_Init();
53     Button_Init();
54
55     const osMessageQueueAttr_t btn_q_attr = { .name = "ButtonQueue" };
56     ButtonQueueHandle = require(osMessageQueueNew(APP_BUTTON_QUEUE_LEN, sizeof(ButtonEvent_t),
57         &btn_q_attr));
58
59     const osMessageQueueAttr_t tx_q_attr = { .name = "TxQueue" };
60     TxQueueHandle = require(osMessageQueueNew(APP_TX_QUEUE_LEN, sizeof(TxItem_t), &tx_q_attr));
```

1. Fonksiyon içindeki static initialized ilk çağrıda false'tur. Bir kez kurulum başladıktan sonra true olur; aynı fonksiyon yeniden çağrılırsa nesneler ikinci defa oluşturulmaz.
2. Timestamp_Init sonucu g_dwt_ok'ya yazılır. RingBuffer_Init tüm kayıt ve sayaçları başlangıç durumuna getirir. Button_Init kenar geçmişi ile olay sayacını sıfırlar.

3. ButtonQueue, ButtonEvent_t boyutunda 8 öge için oluşturulur. TxQueue, TxItem_t boyutunda mevcut modun kapasitesi için oluşturulur. Öğelerin boyutunu tek tek elle yazmak yerine sizeof kullanılır.
4. DUAL_QUEUE etkinse ek BtnTxQueue burada oluşturulur. Ana anlatımdaki LARGE_FIFO dalında bu kod programa girmez.
5. CmdQueue oluşturulur. Bu adım komut alımını başlatmadan önce tamamlanır.

KAYNAK / Src/app_rtos.c / satır 67-74

```

67     const osMessageQueueAttr_t cmd_q_attr = { .name = "CmdQueue" };
68     CmdQueueHandle = require(osMessageQueueNew(APP_CMD_QUEUE_LEN, sizeof(CmdLine_t), &cmd_q_attr));
69
70     const osSemaphoreAttr_t sem_attr = { .name = "TxDoneSem" };
71     TxDoneSemHandle = require(osSemaphoreNew(1, 0, &sem_attr));
72
73     const osEventFlagsAttr_t flags_attr = { .name = "RunFlags" };
74     RunFlagsHandle = require(osEventFlagsNew(&flags_attr));

```

`osSemaphoreNew(1, 0, ...)`: En fazla bir jeton olsun, başlangıçta hiç jeton olmasın. Henüz hiçbir aktarım bitmediği için başlangıç sayısının sıfır olması anlamlıdır. RunFlags ise deney başlatma iznini taşır; başlangıçta RUN_BIT set edilmemiştir.

Sonraki üç `osThreadNew` çağrısı sırasıyla `UartTxTask_Run`, `ButtonTask_Run` ve `TelemetryTask_Run` giriş fonksiyonlarını kaydeder. attr yapıları görevlerin adını, önceliğini ve stack_size değerini verir. Yığınlar sırasıyla 1536, 1024, 1536 bayttır; yerel değişkenler ve fonksiyon çağrılarının geçici çalışma alanıdır. Bu oluşturulma sırası görevlerin sürekli çalışma sırası değildir; çalışma sırasını hazır/bloklu durum ve öncelik belirler.

En son `CmdRx_Init()` çağrılır. Bunun yeri bilinçlidir: alıcı kesmesi bir satırı tamamladığında CmdQueue'ya erişecek. DUAL_QUEUE modunda `UartTxTaskHandle` da bildirim için hazır olmalıdır. Alımı en sona bırakmak bu bağımlılıkları karşılar.

Küçük yardımcılar

`require(handle)`, RTOS nesnesi veya görev oluşturma sonucunun NULL olup olmadığını kontrol eder. NULL ise `Error_Handler` çağrılır; mevcut `Error_Handler` kesmeleri kapatıp sonsuz döngüde kalır. Bu fonksiyon nesneyi oluşturmaz, oluşturma sonucunu denetler. Nesnelerin dinamik belleği RTOS heap'inden gelir; kaynakta heap 24576 bayttır. Bu rehber yeni bir çalışma zamanı bellek doğrulaması yapmaz.

`App_RunElapsedMs()`, `HAL_GetTick()` - `g_run_start_ms` döndürür. TEL mesajındaki deney içi zaman bununla oluşur. Bu kaba kronoloji zamanı, DWT ile ölçülen R'nin yerine kullanılmaz.

Scheduler çalıştığında `TelemetryTask` RUN_BIT bekler; `ButtonTask` ButtonQueue bekler. `UartTxTask` BOOT mesajını oluşturup göndermeyi dener. Böylece bilgisayar saat değerini, DWT kontrol sonucunu ve FW_VERSION="0.3" bilgisini alabilir. DWT kontrolünün false olması bu kodda START'ı engellemez; sonuç BOOT'ta raporlanır.

BÖLÜM 07 / 22

07 PC'den gelen bir bayt nasıl komuta dönüşür?

Komut yolu iki aşamalıdır: **önce satırı eksiksiz toplamak, sonra satırın anlamını çözmek.** app_cmd_rx.c birinci aşamayı yapar. UartTxTask ve Protocol_ParseCommandLine ikinci aşamayı yapar.

app_cmd_rx.c'deki s_rx_byte tek baytlık alım yeridir. s_line tamamlanmamış komut satırını taşır. s_discarding, geçerli bir satır başlangıcına yeniden ulaşana kadar gelenleri atma durumudur. Bunların static olması bir kesmeden sonraki kesmeye kadar birikimin korunmasını sağlar.

Başlatma ve yeniden alıma hazır etme

rearm() bir sonraki bayt için HAL_UART_Receive_IT çağırır. Burada kütüphanenin içini bilmeye gerek yok: uygulama, alınacak bir baytın adresini ve boyutunu bildirir. Alım tamamlandığında kendi HAL_UART_RxCpltCallback fonksiyonu çağırılır. Bu callback UART'ın doğru örnek olduğunu denetler ve CmdRx_HandleByteReceived'e geçer.

CmdRx_Init, len=0 ve discarding=false yapıp rearm çağırır. Her alınan bayt işlendikten sonra da rearm çağırılır. Böylece tek seferlik alım istekleri ardışık bir komut kanalına dönüşür.

KAYNAK / Src/app_cmd_rx.c / satır 24-30

```
24 void CmdRx_HandleByteReceived(void)
25 {
26     char c = (char)s_rx_byte;
27
28     if (c == '\n') {
29         if (!s_discarding && s_line.len > 0u) {
30             /* ISR context: timeout 0. CmdQueue doluysa satir duser; PC tekrar dener. */
```

CmdRx_HandleByteReceived içindeki üç yol

1. Gelen karakter LF (\n) ise satır bitmiştir. Satır atılmıyorsa ve len sıfırdan büyükse CmdQueue'ya kopyalanır. Sonra len sıfırlanır, discarding kaldırılır. Yeni satır için başlangıç durumuna dönülür.
2. Karakter CR (\r) ise biriktirilmez. Böylece CR+LF gönderen bir terminalde CR komutun parçası olmaz.
3. Diğer karakterler, discarding false olduğu ve yer bulunduğu sürece text[len] konumuna yazılır; len artırılır. Kapasite aşılsa discarding=true ve len=0 yapılır. Bundan sonra LF'ye kadar tüm satır atılır.

KAYNAK / Src/app_cmd_rx.c / satır 41-59

```
41     s_line.len = 0u;
42     s_discarding = false;
43 } else if (c != '\r' && !s_discarding) {
44     if (s_line.len < APP_CMD_LINE_MAX_LEN) {
45         s_line.text[s_line.len++] = c;
46     } else {
47         s_discarding = true;
48         s_line.len = 0u;
49     }
50 }
51 rearm();
52 }
53
54 void CmdRx_OnError(void)
55 {
56     s_line.len = 0u;
57     s_discarding = true; /* yarım kalan satır bozuk; bir sonraki LF'den devam */
58     rearm();
59 }
```

Bu davranışın örneği: 64 bayt dolduktan sonra 65'inci normal karakter gelirse satır geçersiz sayılır. Gelen parçanın sonunu ayrı bir komut sanmamak için yalnızca tamponu boşaltmakla kalınmaz, sonraki LF'ye kadar beklenir. Hata callback'inden çağrılan CmdRx_OnError da aynı toparlanma mantığını kullanır; yarım satırı atar ve alımı yeniden açar.

CmdQueue'ya kesmeden Put timeout=0 ile yapılır. Kuyruk doluysa komut satırı kaybolur; bu dosyada bu kayıp için ayrı sayaç artırılmaz. Mevcut tek FIFO modunda dönüş değeri kullanılmaz. DUAL_QUEUE dalında başarılı Put sonrasında UartTxTask'a iş var bayrağı gönderilir.

Kesmeden göreve sınır

Bu aşamada START kelimesi bile yorumlanmaz. Gelen baytların sadece nerede bittiği anlaşılmıştır. CRC, sayısal alanlar ve deney durumunu değiştirme işi kesmeden çıkarılmış olur. Kuyruğa konulan s_line'in kopyası korunur; alıcı bir sonraki satır için aynı s_line nesnesini yeniden kullanabilir.

BÖLÜM 08 / 22

08 Komutu doğrulamak: app_protocol ayrıştırıcısı

PC'den komut biçimi `İÇERİK,HH\n` şeklindedir. HH, içerik üzerinden hesaplanmış CRC-8'in iki büyük harfli onaltılık karakteridir. Örneğin `START,98\n, STOP,66\n, DUMP,22\n`. CRC'ye son virgöl ve LF dahil edilmez. Firmware'den çıkan 64 baytlık çerçevedeki CRC ise ham tek bayttır; iki yönün biçimi aynı değildir.

UartTxTask içindeki `handle_cmd_line()` yerel bir `Command_t` oluşturur ve `Protocol_ParseCommandLine` çağırır. Sonuç `PARSE_CRC_ERROR` ise `cmd_crc_err_count` artırılır. `PARSE_OK` ise `control_handle_command` çağrılır. `PARSE_MALFORMED` ise komut uygulanmaz; bu yolda ayrıca hata yanıtı veya sayaç güncellemesi yoktur.

Protocol_ParseCommandLine sırası

1. out->type önce `CMD_NONE` yapılır. Geçersiz girişte komut seçilmiş gibi görünmesin diye başlangıç durumu hazırlanır.
2. Uzunluk 1..64 aralığında mı bakılır. Satır yerel `buf[65]` içine kopyalanır ve `buf[len]='\0'` yazılır. `strlen(buf)` len'e eşit değilse girişte gömülü NUL vardır; reddedilir.
3. `strchr` ile son virgöl bulunur. Virgülden sonra tam iki karakter olması istenir. Bunların `hex_nibble` ile 0-9 veya A-F olduğu doğrulanır.
4. Bu iki yarım bayt `(hi << 4) | lo` ile tek CRC baytına birleşir. Son virgülden önceki içeriğin CRC'si hesaplanır; eşleşmezse `PARSE_CRC_ERROR` döner.
5. Son virgöl NUL'a çevrilir. Artık elde CRC alanından ayrılmış bir içerik metni vardır. `next_token` ile komut adı ve alanlar sırayla çıkarılır.
6. Ad CFG ise üç sayısal alan beklenir: period, load, scenario. Eksik/fazla alan reddedilir. scenario en fazla 255 olabilir. Doğrulanmış değerler `Command_t`'ye yazılır.
7. Diğer komutlarda ek alan kabul edilmez. `START`, `STOP`, `DUMP` ve `RESET_STATS` adları eşleştirilir. Bilinmeyen ad `PARSE_MALFORMED` olur.

KAYNAK / Src/app_protocol.c / satır 203-219

```

203     char *last_comma = strchr(buf, ',');
204     if (last_comma == NULL || strlen(last_comma + 1) != 2u) {
205         return PARSE_MALFORMED;
206     }
207     uint8_t hi, lo;
208     if (!hex_nibble(last_comma[1], &hi) || !hex_nibble(last_comma[2], &lo)) {
209         return PARSE_MALFORMED;
210     }
211     uint8_t expected_crc = (uint8_t)((hi << 4) | lo);
212     size_t content_len = (size_t)(last_comma - buf);
213     if (Protocol_Crc8((const uint8_t *)buf, content_len) != expected_crc) {
214         return PARSE_CRC_ERROR;
215     }
216     *last_comma = '\0';
217
218     char *cursor = buf;
219     char *name = next_token(&cursor);

```

CRC kontrolü metni virgüllerden bölmeden önce yapılır. `next_token` virgülleri NUL'a çevirdiğinden, önce parçalayıp sonra değiştirilmiş tamponun CRC'sini hesaplamak gelen içeriği doğrulamazdı. Buradaki sıra, doğrulanan baytlarla alınan baytların aynı olmasını sağlar.

Üç küçük yardımcı neden ayrı?

`hex_nibble(c, out)`, tek karakteri 0..15 sayısına çevirir; küçük harf a-f kabul etmez. `next_token(char **cursor)`, bir sonraki virgülü bulur, oraya NUL yazar ve `cursor`'ı sonraki alanın başına taşır. İki yıldızın nedeni, çağıranın tuttuğu işaretçinin de güncellenmesidir. Fonksiyonlar ortak statik ayrıştırma durumu kullanmaz; `cursor` çağırın fonksiyonun yerel değişkenidir.

KAYNAK / Src/app_protocol.c / satır 160-173

```
160 /* Yalnızca rakam; bos, isaretli veya tasan degerleri reddeder. */
161 static bool parse_u32(const char *s, uint32_t *out)
162 {
163     if (s == NULL || *s == '\0') return false;
164     uint32_t v = 0u;
165     for (; *s != '\0'; s++) {
166         if (*s < '0' || *s > '9') return false;
167         uint32_t digit = (uint32_t)(*s - '0');
168         if (v > (UINT32_MAX - digit) / 10u) return false;
169         v = v * 10u + digit;
170     }
171     *out = v;
172     return true;
173 }
```

`parse_u32` sayıyı soldan sağa kurar. Her karakter rakam olmalıdır. `v * 10 + digit` yapmadan önce taşma kontrol edilir: `v, (UINT32_MAX - digit) / 10` sınırından büyükse yeni değer 32 bite sığmayacaktır. Örneğin "200" için ara değerler 2, 20, 200 olur. Boş metin, eksi işareti, artı işareti veya harf kabul edilmez.

Bu fonksiyon periyodun uygulama için uygunluğunu seçmez; yalnızca biçim ve sayısal sınırları doğrular. CFG'deki minimum 10 ms kuralı, ayrıştırmadan sonraki kontrol katmanındadır. Böylece "mesaj doğru yazılmış mı?" ve "bu durumda uygulanabilir mi?" iki ayrı karar olarak kalır.

BÖLÜM 09 / 22

09 Deneyin kumandası: CFG, START, STOP, DUMP

`control_handle_command()` `app_tasks.c` içinde static bir yardımcıdır. `UartTxTask` tarafından çağrıldığı için komutlar aynı görevde sırayla işlenir. `g_cfg`'nin yazılması, kayıtların sıfırlanması ve UART'tan dump gönderimi bu kontrol noktasında birleşir.

Komut	Kodun kabul koşulu	Yaptığı iş
CFG	<code>g_running</code> false	Periyodu en az 10 yapar; iterasyonu ve senaryo etiketini yazar
START	<code>g_running</code> false	Kayıtları/sayaçları ve olay kimliğini sıfırlar; başlangıç zamanını alır; çalışmayı açar
STOP	<code>g_running</code> true	<code>RUN_BIT</code> 'i temizler; <code>g_running</code> 'i false yapar
DUMP	Çalışma kapalı ve ilgili TX kuyrukları boş	REC kayıtlarını, STAT'ı ve DUMP_END'i doğrudan gönderir
RESET_STATS	<code>g_running</code> false	Ring buffer ve olay kimliğini sıfırlar; çalışmayı açmaz

Koşulu sağlanmayan komut, geçerli CRC'si olsa da ilgili işlemi yapmadan biter. Burada ayrıca ACK/NAK üreten bir dal yoktur. CFG sırasında çalışma açıksa ayarlar değişmez. Aynı START çalışırken yeniden gelirse mevcut deney sıfırlanmaz.

START'ta neden önce durum, sonra bayrak?

KAYNAK / `Src/app_tasks.c` / satır 119-137

```

119     case CMD_START:
120         if (!g_running) {
121             RingBuffer_ResetForScenario();
122             Button_ResetForScenario();
123             g_run_start_ms = HAL_GetTick();
124             g_running = true;                                /* önce bayrak degisken... */
125             (void)osEventFlagsSet(RunFlagsHandle, RUN_BIT); /* ...sonra TelemetryTask uyanır */
126         }
127         break;
128
129     case CMD_STOP:
130         if (g_running) {
131             /* Sıra önemli: önce event flag temizlenir. Tersisi olursa yüksek
132              * öncelikli TelemetryTask araya girip bayrak hala set iken
133              * bekleme dongusunda donerek UartTxTask'i ac bırakabilir. */
134             (void)osEventFlagsClear(RunFlagsHandle, RUN_BIT);
135             g_running = false;
136         }
137         break;

```

Önce eski kayıtlar ve olay numarası sıfırlanır. Sonra `g_run_start_ms` alınır. Ardından `g_running=true` yazılır. En son `RUN_BIT` set edilir. `RUN_BIT` set edilmesi yüksek öncelikli `TelemetryTask`'ı hemen hazır hâle getirebilir. O görev çalışmaya geçtiğinde `g_running`'in de true olmasını sağlamak için bu sıralama vardır.

STOP'ta sıra ters yöndedir: önce RUN_BIT kaldırılır, sonra g_running=false yapılır. RUN_BIT açık kalırken yüksek öncelikli görev dış beklemesine dönerse osFlagsNoClear nedeniyle bekleme hemen çözülür. Kaynak yorumunun korumak istediği geçiş budur. TelemetryTask içindeki ek osDelay(1) kontrolü de bu geçişte CPU'yu bırakmayı sağlar.

g_running ve RUN_BIT neden ikisi birden var? g_running, ISR ve çalışan döngülerin hızlıca kontrol ettiği durumdur. RUN_BIT ise henüz deney yokken TelemetryTask'ın sürekli sorgulamak yerine RTOS içinde uyumasını sağlar. Birincisi koşul, ikincisi beklenebilir bildirim/durum mekanizmasıdır.

STOP'un gerçek kapsamı

STOP yeni kabul edilen buton olaylarını ve yeni telemetri üretimini kapatır. Önceden ButtonQueue veya TX kuyruğuna teslim edilmiş işleri temizlemez; etkin UART gönderimini iptal etmez. ButtonTask zaten aldığı olayın işlenmesine devam edebilir. UartTxTask da kuyruktaki mesajları tüketir. Bu yüzden STOP ile DUMP aynı işlem değildir.

DUMP koşulu açıkça yalnızca g_running ve TX kuyruğu sayısını kontrol eder; ButtonQueue için ayrı bir boşluk denetimi yoktur. UartTxTask komutu işlerken aynı görevde önceki send_item çağrısı bitmiştir, fakat koşulun kendisi tüm kaynakların boş olduğunu sınavan genel bir kontrol değildir. Bu, kodda yazılı kabul şartının tam kapsamıdır.

START ve RESET_STATS ring kayıtlarını sıfırlar; RTOS kuyruklarını sıfırlayan bir çağrı içermez. Öğretici akış bu nedenle CFG -> START -> deney -> STOP -> gönderimlerin boşalması -> DUMP şeklindedir. Bu rehber komutların var olmayan etkilerini varsaymaz.

BÖLÜM 10 / 22

10 Buton kenarı: app_button.c

Donanım tarafında buton pini hem yükselen hem düşen kenarda kesme oluşturacak biçimde ayarlanmış. Uygulama açısından çağrı zinciri şudur: EXTI15_10_IRQHandler -> HAL'in EXTI dağıtımı -> uygulamanın HAL_GPIO_EXTI_Callback fonksiyonu -> Button_HandleEXTI_Callback, GPIO_Pin değerinin USER_BUTTON_PIN olduğunu denetler. HAL kodunun içeri açmadan uygulamaya giriş yerini bulmuş oluyoruz.

Button_Init üç durumu sıfırlar: s_last_edge_ms son görülen kenarın zamanı, s_edge_seen daha önce kenar görülüp görülmediği, s_event_counter olay sayacıdır. Button_ResetForScenario yalnızca olay sayacını sıfırlar. Debounce geçmişini her START'ta temizlenmez; fiziksel butonun kenar geçmişi senaryolar arasında devam eder.

Filtre önce zamanı alıyor, sonra karar veriyor

KAYNAK / Src/app_button.c / satır 24-41

```

24 void Button_HandleEXTI(void)
25 {
26     uint32_t t_entry = Timestamp_Now(); /* ISR girişi: kabul edilirse t0 bu */
27     uint32_t now_ms = HAL_GetTick();
28
29     bool quiet = !s_edge_seen || (uint32_t)(now_ms - s_last_edge_ms) >= APP_DEBOUNCE_MS;
30     s_last_edge_ms = now_ms;
31     s_edge_seen = true;
32
33     bool pressed = HAL_GPIO_ReadPin(USER_BUTTON_PORT, USER_BUTTON_PIN) == USER_BUTTON_PRESSED_STATE;
34     if (!quiet || !pressed || !g_running) {
35         return;
36     }
37
38     uint16_t event_id = (uint16_t)(s_event_counter + 1u);
39     s_event_counter = event_id;
40     uint8_t scenario_id = g_cfg.scenario_id;
41     uint32_t t0_abs_ms = now_ms - g_run_start_ms;

```

İlk satırdaki t_entry olası t0'dır. Sonradan kenar kabul edilmezse bu değer kullanılmaz. Önce damga almak, kabul edilen olayın zamanını filtre hesaplarından sonraya kaydırmaz. Ardından milisaniye saati debounce ve deney içi kronoloji için okunur.

quiet şu soruyu sorar: "Hiç kenar görmedik mi veya önceki herhangi bir kenardan beri en az 30 ms geçti mi?" Sonra, kabul edilip edilmeyeceğinden bağımsız olarak s_last_edge_ms güncellenir ve s_edge_seen true olur. Böylece reddedilen sekme ve bırakma kenarları da sessizlik penceresini yeniden başlatır.

pressed pinin o anda basılı seviyede olmasıdır. Koşullardan biri sağlanmazsa hemen çıkılır: yeterince sessizlik yoksa, pin basılı değilse veya deney kapalıysa olay üretilmez. Böyle bir kenar için event_id artırılmaz ve ring kaydı ayrılmaz.

Örnek: 100 ms'de kabul edilen basıştan sonra 103 ve 107 ms'de sekme kenarları gelsin. Bu kenarlar son kenar zamanını yeniler. 120 ms'deki yeni yüksek seviye, 107'den yalnızca 13 ms sonra olduğu için reddedilir. Bu filtre "yeni kenarı gördükten sonra 30 ms bekle" algoritması değildir; **kenardan önceki sessizliği** kullanır. Ayrıca yalnızca zaman geçmesi olayı sonradan otomatik üretmez; yeni bir kesme gerekir.

Kabul edilen kenar bir olaya dönüşüyor

KAYNAK / Src/app_button.c / satır 43-54

```
43     ButtonEvent_t evt = {
44         .t0          = t_entry,
45         .t0_abs_ms   = t0_abs_ms,
46         .event_id    = event_id,
47         .ring_index  = RingBuffer_Alloc(event_id, scenario_id, t_entry, t0_abs_ms),
48         .scenario_id = scenario_id,
49     };
50
51     /* ISR context: timeout 0 zorunlu. ButtonQueue (8) dolu ise DROP. */
52     if (osMessageQueuePut(ButtonQueueHandle, &evt, 0u, 0u) != osOK) {
53         RingBuffer_CloseDropIsr(evt.ring_index, event_id);
54     }
```

event_id, önceki sayaca bir eklenerek atanır. İlk senaryo olayının kimliği 1 olur. 16 bit olduğu için sonsuza kadar artan bir kimlik değildir. scenario_id o anki g_cfg'den alınır. t0_abs_ms, deney başlangıcından bu kenara kadar geçen milisaniyedir.

RingBuffer_Alloc önce bir RAM kaydı açar ve onun indeksini döndürür. **Neden queue'ya koymadan önce kayıt?** Çünkü ButtonQueue dolu olsa bile kabul edilmiş basışın kaybolduğunu kayıt altına almak istiyoruz. Slot ayrıldıktan sonra Put başarısızsa RingBuffer_CloseDropIsr bu olayı DROP yapabilir.

ButtonQueue Put'un dört argümanı sırasıyla hedef kuyruk, kopyalanacak olayın adresi, mesaj önceliği 0 ve bekleme 0'dır. ISR boş yer bekleyemez. Başarılıysa olayın kopyası ButtonTask'a teslim edilmek üzere saklanır. Başarısızsa buton kuyruğu kayıp sayacı artar. Fonksiyon burada biter; metin biçimlendirme ve UART aktarımı bu kesmede yapılmaz.

BÖLÜM 11 / 22

11 ButtonTask: olayı yanıt mesajına çevirmek

ButtonTask_Run sürekli ButtonQueue'dan olay bekler. Get çağrısının osWaitForever parametresi, olay yokken bloklu beklemesini sağlar. İşlemci boş bir kuyruğu sürekli sorgulayarak meşgul edilmez. Get beklenmedik biçimde başarısız dönerse continue ile yeniden beklemeye geçilir.

KAYNAK / Src/app_tasks.c / satır 220-236

```

220     for (;;) {
221         ButtonEvent_t evt;
222         if (osMessageQueueGet(ButtonQueueHandle, &evt, NULL, osWaitForever) != osOK) {
223             continue;
224         }
225         uint32_t t1 = Timestamp_Now(); /* t1: olay alındıktan hemen sonra */
226
227         /* Yanıt üretimi (S2 = t2 - t1 bu blogu olcer). t2 henuz bilinmedigi
228          * icin yanita konmaz; tam kayıt dump ile gelir (SPEC.md S6). */
229         TxItem_t item;
230         item.type      = MSG_TYPE_BTN;
231         item.event_id   = evt.event_id;
232         item.ring_index = evt.ring_index;
233         (void)Protocol_BuildBTN(item.frame, evt.event_id, evt.scenario_id, evt.t0_abs_ms,
234                                Timestamp_DeltaToUs(evt.t0, t1));
235
236         uint32_t t2 = Timestamp_Now(); /* t2: kuyruğa bırakma çağrısından hemen önce */

```

Get başarılı döner dönmez t1 alınır. Bu satıra gelindiğinde evt artık görevin yerel kopyasıdır. t1'in burada olması, kesmeden göreve ulaşma süresini yanıt hazırlama süresinden ayırır.

Sonra TxItem_t hazırlanır: type BTN olur; event_id ve ring_index olaydan kopyalanır. Aynı olaya ait bağın korunması, gönderim bitince hangi RAM kaydının kapatılacağını belirler. Protocol_BuildBTN, event_id, scenario_id, t0_abs_ms ve t1-t0'ın mikrosaniye karşılığını kullanarak frame alanını doldurur.

Çerçeve bittikten sonra t2 alınır. Böylece t2-t1 alanları hazırlamayı, snprintf'i ve CRC hesabını kapsar. **t2'nin hazır mesajın içine yazılmamasının nedeni zaman sırasıdır:** mesajın oluşturulma bitişini ölçmek için t2, mesaj tamamlandıktan sonra alınır. t4 zaten daha sonra, gönderim bitince bilinecektir. Canlı BTN hızlı yanıtı taşır; tam ölçüm kaydı deney sonu REC ile gelir.

Hazır mesajı TX yoluna bırakmak

KAYNAK / Src/app_tasks.c / satır 247-254

```

247         osStatus_t st = osMessageQueuePut(TxQueueHandle, &item, 0u, APP_TX_QUEUE_SEND_TIMEOUT_MS);
248
249         /* ButtonTask, UartTxTask'ten yuksek oncelikli oldugu icin bu yazma,
250          * UartTxTask'in ayni kayda t3 yazmasindan once tamamlanir. */
251         RingBuffer_SetT1T2(evt.ring_index, evt.event_id, t1, t2);
252         if (st != osOK) {
253             RingBuffer_CloseDropTx(evt.ring_index, evt.event_id);
254         }

```

Mevcut tek FIFO dalında ButtonTask, TxQueue doluysa en çok 200 tick bekler. Kuyrukta yer varsa hemen kopyalar. Bu çağrıda bloklanırsa t2 zaten alınmış olduğundan yer bekleme süresi A3=t3-t2 içinde kalır; A2'ye yazılmaz.

Put çağrısından sonra RingBuffer_SetT1T2 ile yerel damgalar RAM kaydına yazılır. Damgaları önce yerel değişkende almak, kayıt yazma maliyetini t1-t2 yanıt hazırlama aralığına koymaz. Fakat kayıt yazmanın maliyeti sihirli biçimde yok

olmaz; sonraki akışa, dolayısıyla toplam süreye etkisi olabilir. Burada anlatılan, damganın alındığı yer ile kayda yazıldığı yerin farklı olmasıdır.

ButtonTask UartTxTask'tan yüksek önceliklidir. Bu yüzden başarılı teslim sonrasında bu kısa kayıt yazımı, düşük öncelikli göndericinin aynı ögenin aktarımını iletmesinden önce yapılır. Üst öncelikli telemetri araya girebilir; fakat o da UartTxTask'a göre daha önceliklidir.

Put başarısızsa kayıt DROP olarak kapatılır ve tx_queue_drop_count artar. Başarılıysa ButtonTask UART'ın bitmesini beklemeyiz; bir sonraki buton olayını almak üzere döngüye döner. Görevi yanıt üretip teslim etmektir. Gönderim sonucu daha sonra UartTxTask tarafından aynı event_id/ring_index çiftiyle kayda işlenir.

DUAL_QUEUE dalında aynı hazır BTN ayrı BtnTxQueue'ya timeout=0 ile konur. Başarıda göndericiye thread flag set edilir, başarısızlıkta yine DROP yazılır. İki modun bütün farkları 18. bölümde bir araya getiriliyor.

BÖLÜM 12 / 22

12 TelemetryTask: periyot ve yapay CPU yükü

TelemetryTask_Run hem periyodik durum mesajı üretir hem de görevlere CPU beklemesi oluşturacak ayarlı hesap yapar. En yüksek uygulama önceliği burada bilinçlidir: hesap döngüsü çalışırken daha düşük öncelikli ButtonTask ve UartTxTask gecikebilir. ISR'ların görevlerden ayrı bir öncelik düzeni vardır; "en yüksek görev" demek "kesmeler de çalışmaz" demek değildir.

Dış döngü deneyi, iç döngü periyotları yönetir

KAYNAK / Src/app_tasks.c / satır 266-289

```

266     for (;;) {
267         (void)osEventFlagsWait(RunFlagsHandle, RUN_BIT, osFlagsWaitAny | osFlagsNoClear,
osWaitForever);
268         if (!g_running) {
269             osDelay(1u); /* STOP gecisinin ortasi: donerek alt gorevleri ac birakma */
270             continue;
271         }
272
273         uint32_t seq = 0u;
274         uint32_t wake = osKernelGetTickCount(); /* her START'ta tazele: eski deger ani TEL patlamasi
uretir */
275
276         while (g_running) {
277             uint32_t period = g_cfg.period_ms;
278             wake += period;
279             uint32_t now = osKernelGetTickCount();
280             if ((int32_t)(wake - now) <= 0) {
281                 /* Tasma (yuk >= periyot): kaybedilen periyotlari telafi etmeye
282                  * calisma, yeniden hizala ve yine de bir periyot uyu. Aksi
283                  * halde en yuksek oncelikli gorev hic bloklanmaz ve tum alt
284                  * gorevleri (STOP'u isleyecek olan UartTxTask dahil) ac birakir. */
285                 wake = now + period;
286             }
287             (void)osDelayUntil(wake);
288             if (!g_running) {
289                 break;

```

Dış döngü RUN_BIT bekler. osFlagsNoClear seçeneği, bekleme karşılandı diye RUN_BIT'in otomatik temizlenmesini engeller; bit deney boyunca açık kalır ve STOP tarafından kaldırılır. Uyanınca g_running ayrıca kontrol edilir. Çalışma kapalıysa osDelay(1) ile kısa süre bloklanıp başa dönlür.

Her yeni çalışmada seq=0 ve wake=şimdiki kernel tick yapılır. Önceki deneyin zaman çizelgesi kullanılmaz. İç döngü g_running true oldukça sürer. period alınır, wake += period yapılır ve osDelayUntil(wake) ile mutlak bir sonraki tarihe kadar beklenir. İlk TEL, START anında hemen üretilmez; normal akışta önce bir periyot beklenir.

Örnek: başlangıç tick'i 1000, periyot 50 ise hedefler 1050, 1100, 1150... olur. Her döngünün sonundan itibaren yeniden 50 saymak yerine hedef tarihler ilerletilir. Kodun wake - now kontrolü, hedef zaten geçmişse eski periyotları peş peşe telafi etmeye çalışmaz; hedefi now+period yapar. Böylece yük uzun sürse de yüksek öncelikli görev tekrar bloklanır ve diğer görevlere çalışma fırsatı verir.

osDelayUntil dönüşünden sonra g_running yeniden kontrol edilir. Çünkü görev uyurken STOP işlenmiş olabilir. Bu durumda yeni bir hesap yükü ve TEL oluşturmadan iç döngüden çıkılır.

Yük döngüsünün her satırı

KAYNAK / Src/app_tasks.c / satır 292-306

```

292      /* Sabit iterasyonlu yapay CPU yuku; suresi kalibrasyon için TEL'de raporlanır. */
293      uint32_t load_start = Timestamp_Now();
294      volatile uint32_t acc = 0u;
295      for (uint32_t i = 0u; i < g_cfg.load_iter; i++) {
296          acc += (i * 2654435761u) ^ (acc << 1);
297      }
298      uint32_t load_us = Timestamp_DeltaToUs(load_start, Timestamp_Now());
299
300      RingCounters_t counters = RingBuffer_GetCounters();
301      TxItem_t item;
302      item.type = MSG_TYPE_TEL;
303      item.event_id = 0u;
304      item.ring_index = RING_INDEX_NONE;
305      (void)Protocol_BuildTEL(item.frame, ++seq, g_cfg.scenario_id, App_RunElapsedMs(),
306                             g_cfg.period_ms, g_cfg.load_iter, load_us, &counters);

```

load_start hesap döngüsü öncesindeki cycle değeridir. acc, sonucu tutulacak volatile birikendir. i sıfırdan load_iter-1'e kadar gider. Her turda çarpma, kaydırma, XOR ve toplama yapılır. Amaç anlamlı bir sensör hesabı üretmek değil, tekrarlanabilir sayıda işlem yaptırmaktır. volatile, kullanılmayan hesap sonucunun derleyici tarafından tamamen silinmesini engeller; görevi uyutmaz.

load_iter=0 ise döngü gövdesi çalışmaz. Ölçülen load_us yine iki zaman okuması arasındaki küçük maliyeti içerebilir. İterasyon sayısı doğrudan milisaniye değildir; süre bu döngüden sonra Timestamp_DeltaToUs ile ölçülür. Kesmeler araya girerse geçen zaman ölçümüne dahil olabilir; load_us yalnızca saf aritmetik cycle sayısı diye okunmamalıdır.

RingBuffer_GetCounters bir sayaç kopyası döndürür. TEL ögesinde type TEL, event_id 0 ve ring_index RING_INDEX_NONE seçilir. RING_INDEX_NONE, "bu mesajın kapatılacak bir buton kaydı yok" işaretidir. Her TEL için ring slotu ayrılmaz. Protocol_BuildTEL'e ++seq gönderildiği için ilk sıra numarası 1'dir.

Mevcut modda TEL, BTN ile aynı TxQueue'ya 200 tick'e kadar bekleyerek bırakılır. Teslim başarısızsa tel_drop_count artar. TEL başarısızlığı bir buton kaydını DROP yapmaz. Sonraki periyot için döngü sürer. DUAL_QUEUE dalında TEL Put timeout=0'dır; başarıda thread flag, başarısızlıkta aynı telemetry kayıp sayacı kullanılır.

Yük hangi ölçüme yansiyabilir?

Buton kesmesi TelemetryTask yükü sırasında olursa olay kuyrukta bekleyebilir ve A1 büyüyebilir. Telemetry, BTN hazırlanırken hazır olursa A2'ye; BTN gönderime ulaşmayı beklerken çalışırsa A3'e de süre ekleyebilir. DMA çalışırken CPU yükü sürse bile donanım aktarımı bağımsız ilerler. Bu nedenle tek bir "yük süresi" bütün aşamalara aynı biçimde eklenmez; damgaların alındığı anlar belirleyicidir.

BÖLÜM 13 / 22

13 UartTxTask: UART'ın tek sahibi

UartTxTask_Run başlarken BOOT çerçevesini oluşturur ve send_meta ile gönderir. Daha sonra ana döngüde önce CmdQueue'yu, sonra TX yolunu işler. UART üzerinden canlı mesaj, açılış mesajı ve dump gönderen uygulama çağrıları bu görev bağlamında birleşir.

KAYNAK / Src/app_tasks.c / satır 183-193

```
183     uint8_t boot[APP_FRAME_SIZE];
184     (void)Protocol_BuildBOOT(boot, SystemCoreClock, g_dwt_ok, FW_VERSION);
185     send_meta(boot);
186
187     for (;;) {
188         CmdLine_t line;
189         while (osMessageQueueGet(CmdQueueHandle, &line, NULL, 0u) == osOK) {
190             handle_cmd_line(&line);
191         }
192
193         TxItem_t item;
```

CmdQueue için timeout=0 kullanılır. while(Get == osOK) eldeki komutları sırayla işler; komut kalmayınca çıkar. Bu sıra sayesinde hazır komutlar yeni bir TX ögesi alınmadan önce ele alınır. Ancak UartTxTask etkin uart_tx_send içindeyken ana döngüye dönemez; STOP satırının RX tarafından alınmış olması, kontrol fonksiyonunda hemen uygulanmış olması değildir.

Tek FIFO dalında osMessageQueueGet(TxQueueHandle, &item, NULL, 20u) vardır. Veri gelirse send_item çağrılır. Veri yoksa görev 20 tick'e kadar bloklanıp döngü başına döner; bu kez yeniden komutlara bakar. 20 tick kullanılması, boş TX kuyruğunda sonsuza kadar uyuyup yalnızca CmdQueue'da bekleyen komutu görmeme durumunu önler.

Kaynak yorumu "en geç 20 ms" ifadesini kullanır. Tam davranış şudur: **20 tick, bu Get çağrısındaki bekleme sınırıdır.** CPU'yu daha yüksek öncelikli görevlerin kullanması, devam eden aktarım veya dump süresi genel komut yanıtını ayrıca etkileyebilir. Bu sayı uçtan uca STOP garantisi değildir.

send_item: mesaj ile ölçüm kaydını bağlamak

KAYNAK / Src/app_tasks.c / satır 56-76

```
56 static void send_item(const TxItem_t *item)
57 {
58     uint32_t t3 = 0u, t4 = 0u;
59     TxResult_t r = uart_tx_send(item->frame, &t3, &t4);
60
61     if (item->ring_index == RING_INDEX_NONE) {
62         if (r != TX_OK) {
63             RingBuffer_IncTelTxFail();
64         }
65         return;
66     }
67
68     /* Kayıt yazımı olcum zincirinin disinda (gonderimden sonra) yapilir. */
69     RingBuffer_SetT3(item->ring_index, item->event_id, t3);
70     switch (r) {
71         case TX_OK:          RingBuffer_CloseSuccess(item->ring_index, item->event_id, t4); break;
72         case TX_TIMEOUT:     RingBuffer_CloseTimeout(item->ring_index, item->event_id);      break;
73         case TX_START_ERR:
74         case TX_DMA_ERR:
75         default:             RingBuffer_CloseTxErr(item->ring_index, item->event_id);        break;
76     }
```

Yerel t3 ve t4 sıfırlanır; uart_tx_send çerçeveyi gönderip sonucu döndürür. ring_index RING_INDEX_NONE ise ögenin buton kaydı yoktur. TEL gönderimi başarısızsa tel_tx_fail_count artırılır ve çıkılır.

BTN ögesinde önce RingBuffer_SetT3 yapılır. Sonuca göre CloseSuccess, CloseTimeout veya CloseTxErr çağrılır. TX_START_ERR ile TX_DMA_ERR aynı kayıt durumuna, TX_ERR'a gider. Başlatma başarısızlığında bile t3, başlatma denemesinden önce alınmıştır; dolayısıyla kayıta t3 bulunabilir ama t4 bulunmaz.

T3 ve kapanışın uart_tx_send dönüşünden sonra yazılması, gerçek t3/t4'ün o anda alındığı anlamına gelmez. t3 yerel değişkende, t4 callback'in paylaşılan değişkeninde zaten alınmıştır. Bu fonksiyon onların **kayda geçirilmesini** yapar. Tamamlanma sonrası UartTxTask'ın CPU beklemesi t4 değerini değiştirmez.

send_meta: BOOT ve dump için ortak yol

send_meta, uart_tx_send'i çağırır; başarısız olursa bir kez daha dener. Toplam en fazla iki deneme vardır. İkinci dönüş değeri kullanılmaz. BTN/TEL send_item yolunda böyle bir yeniden deneme yoktur. Metadata çerçeveleri normal TxQueue'ya konulmaz; UartTxTask tarafından doğrudan aynı gönderim yordamına verilir. Böylece ayrıca tüketilecek bir TX ögesi yaratılmaz.

BÖLÜM 14 / 22

14 DMA gönderimi ve tamamlanma callback'leri

Bir gönderimi başlatmakla gönderimin bitmesi farklı zamanlardır. `uart_tx_send()` bu iki an arasındaki uygulama düzenini kurar. `HAL_UART_Transmit_DMA` çağrısının amacı, 64 baytı donanıma aktarılacak üzere başlatmaktır; HAL'in register işlemleri bu rehberin dışında kalır.

Önce belleğin ömrü garanti edilir

`s_tx_buffer[64]` dosya düzeyinde static bir dizidir ve 32 bayt hizalanmıştır. DMA fonksiyon çağrısından sonra da o belleği okur. `UartTxTask` tek gönderim sahibi olduğundan ve tamamlanmadan sonraki mesaja geçmediğinden, bu dizi aktarım boyunca geçerli kalır.

KAYNAK / `Src/app_tasks.c` / satır 27-43

```
27 static TxResult_t uart_tx_send(const uint8_t frame[APP_FRAME_SIZE], uint32_t *t3, uint32_t *t4)
28 {
29     memcpy(s_tx_buffer, frame, APP_FRAME_SIZE);
30 #if defined(__DCACHE_PRESENT) && (__DCACHE_PRESENT == 1U)
31     if ((SCB->CCR & SCB_CCR_DC_Msk) != 0u) {
32         SCB_CleanDCache_by_Addr((uint32_t *)s_tx_buffer, (int32_t)sizeof(s_tx_buffer));
33     }
34 #endif
35     /* Onceki bir timeout'tan kalmis gec gelen TC jetonunu temizle. */
36     while (osSemaphoreAcquire(TxDoneSemHandle, 0u) == osOK) {
37     }
38     g_tx_dma_error = false;
39
40     *t3 = Timestamp_Now(); /* t3: UART baslatma cagrisindan hemen once */
41     if (HAL_UART_Transmit_DMA(&APP_UART_HANDLE, s_tx_buffer, APP_FRAME_SIZE) != HAL_OK) {
42         return TX_START_ERR;
43     }
```

Önce `frame`, `s_tx_buffer`'a kopyalanır. Veri ön belleği gerçekten açıksa ilgili bölge temizlenir; böylece DMA'nın RAM'den okuyacağı içerik hazırlanır. 32 bayt hizalama ve 64 bayt boyut bu temizleme alanını düzenler. Mevcut main yalnızca I-Cache'i açar; burada anlatılan D-Cache temizliği koşullu korumadır.

Sonra `osSemaphoreAcquire(..., 0)` döngüsü önceki aktarım/zaman aşımından kalmış bir bildirim jetonunu tüketir. Kuyruğu boşaltmıyor, tamamlanma işaretini temizliyor. Ardından `g_tx_dma_error=false` yapılır. Böylece yeni aktarım öncekinin hata bilgisiyle başlamaz.

Bu hazırlıklardan sonra `t3` alınır ve DMA başlatılır. Kopyalama ve cache temizleme `t3` öncesindedir; buton yanıtında bunların süresi A3 içinde bulunabilir. Başlatma çağrısı `HAL_OK` dönmezse `TX_START_ERR` hemen döner; tamamlanma beklenmez.

Başlatınca görev uyur, donanım çalışır

KAYNAK / Src/app_tasks.c / satır 45-54

```

45     if (osSemaphoreAcquire(TxDoneSemHandle, APP_TX_TC_TIMEOUT_MS) != osOK) {
46         (void)HAL_UART_AbortTransmit(&APP_UART_HANDLE);
47         return TX_TIMEOUT;
48     }
49     if (g_tx_dma_error) {
50         return TX_DMA_ERR;
51     }
52     *t4 = g_tx_done_cycles; /* t4: TC callback icinde (ISR) alindi, gorev uyanma gecikmesi dahil
degil */
53     return TX_OK;
54 }
```

osSemaphoreAcquire, en çok 100 tick tamamlanma bildirimi bekler. Bu sırada UartTxTask bloklanır. DMA'nın bayt taşınması için görev döngüsünün sürekli çalışması gerekmez. Bildirim zamanında gelmezse HAL_UART_AbortTransmit çağırılır ve TX_TIMEOUT döner.

Bildirim gelmişse bu tek başına başarı demek değildir. DMA hata callback'i de aynı semaphore'u bırakır. Bu yüzden hemen ardından g_tx_dma_error kontrol edilir. True ise TX_DMA_ERR döner. Hata yoksa g_tx_done_cycles t4'e kopyalanır ve TX_OK döner.

Tamamlanma callback'inde sıra neden böyle?

KAYNAK / Src/app_hal_callbacks.c / satır 17-27

```

17 void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
18 {
19     if (huart->Instance != APP_UART_INSTANCE) {
20         return;
21     }
22     /* t4: TC (son bit gonderildi) isleniyor. Burada alinir; UartTxTask en
23      * dusuk oncelikli oldugu icin uyandiginda alinsaydi, araya giren ust
24      * gorevlerin (CPU yuku dahil) suresi S4'e eklenirdi. */
25     g_tx_done_cycles = Timestamp_Now();
26     (void)osSemaphoreRelease(TxDoneSemHandle);
27 }
```

Önce UART örneği kontrol edilir; başka UART'ın olayı bu uygulamaya ait sayılmaz. Ardından t4 alınır ve en son semaphore bırakılır. Veri önce yazılır, o veriyi okuyacak görev sonra uyandırılır. t4'ü UartTxTask uyandığında almak, düşük öncelikli görevin CPU'yu geri alma beklemesini de gönderim süresine katardı. Mevcut kod bitiş kesme bağlamında damgalar.

Normal TX DMA yolunda DMA aktarımının kendi tamamlanması, UART'taki son bitin telden çıkmış olmasıyla aynı olay değildir. Bu uygulamanın t4'ü HAL_UART_TxCpltCallback'teki UART tamamlanma yoluna bağlıdır. Kayıttaki bitiş sıfır DMA belleği okumayı bitirdi diye erkene çekmiyoruz.

Diğer üç callback'in işi

Uygulamanın yazdığı callback	Yaptığı işlem	Sonraki bağlantı
HAL_UART_ErrorCallback	Doğru UART'ta DMA hata bitini ayırır	g_tx_dma_error=true; semaphore bırakılır
Aynı hata callback'inin RX dalı	ORE/FE/NE/PE alım hata bitlerini ayırır	CmdRx_OnError ile yarım satır atılır, alım yeniden kurulur
HAL_UART_RxCpltCallback	Doğru UART'ın baytı tamamlandı mı bakar	CmdRx_HandleByteReceived
HAL_GPIO_EXTI_Callback	Doğru buton pini mi bakar	Button_HandleEXTI

DMA ve RX hata kontrolleri iki ayrı if bloğudur; aynı hata kodunda uygun bitler varsa iki yol da işlenebilir. RX hatası dalı tek başına TX semaphore'unu bırakmaz. app_hal_callbacks.c'deki bu gövdeler uygulamanın kendi kodudur. HAL'deki weak varsayılan gövdelerin yerine link aşamasında bağlanırlar; bu yüzden uygulamada normal bir çağrı satırı görmeseniz de donanım olayıyla çalışırlar.

Mevcut hızda fiziksel süre

Kaynak 921600 baud ve 8N1 seçiyor. Bir bayt için 1 başlangıç + 8 veri + 1 bitiş = 10 bit gönderilir. 64 bayt için ideal hat süresi $64 \times 10 / 921600 = 0,00069444$ s, yaklaşık **694,44 us** olur. Bu hesap yalnızca hat üzerindeki ideal süredir; ölçülen A4 yazılım başlatma ve callback gecikmesini de içerebilir. Kaynak yorumlarındaki yaklaşık 5,56 ms, 115200 baud hesabına aittir.

BÖLÜM 15 / 22

15 Ring buffer: bir olayın parça parça tamamlanan kaydı

Queue, yapılacak işi taşır ve öge alınınca kuyruktan çıkar. Ring buffer ise olmuş olayın kaydını deney sonuna kadar saklar. Buton yanıtı UART'tan çıkmış olsa bile onun t0..t4 değerleri RAM'de kalır. Aynı olayın hem kuyrukta hem ring'de bulunmasının nedeni bu farklı ömürlerdir.

Kayıt alanları

KAYNAK / Inc/app_ring_buffer.h / satır 34-42

```
34 typedef struct {
35     uint32_t t0, t1, t2, t3, t4; /* DWT cycle */
36     uint32_t t0_abs_ms; /* deney basından (START) itibaren ms */
37     uint16_t event_id;
38     uint8_t scenario_id;
39     uint8_t status; /* RecordStatus_t */
40     uint8_t ts_mask; /* REC_TS_* : hangi t'ler gercekten ölçüldü */
41     bool valid;
42 } EventRecord_t;
```

t0..t4 ham cycle değerleridir. t0_abs_ms grafikte deneyin hangi anına bakıldığını anlatır. event_id ve scenario_id kaydın kimliğidir. status sonucun sayısal adıdır. valid, slotun gerçekten ayrılmış olduğunu belirtir. ts_mask ise t1..t4 alanlarından hangilerinin ölçüldüğünü söyler.

Bir damganın sıfır olması "ölçülmedi" demek değildir; cycle sayacı gerçekten sıfırdan geçmiş olabilir. Bu yüzden ölçülme bilgisi değerinden ayrı tutulur. T1 biti 1, T2 biti 2, T3 biti 4, T4 biti 8'dir. t1 ve t2 yazılınca maske 3 olur; t3 eklenince 7, t4 eklenince 15 olur. |= daha önce set edilmiş bitleri koruyarak yeni bit ekler.

Başlatma ile senaryo sıfırlama farkı

RingBuffer_Init diziyi, toplam ayrılan kayıt sayısını ve sayaçları sıfırlar. Başlangıç kurulumunda çağrılır. RingBuffer_ResetForScenario aynı sıfırlamayı görev içinden kritik bölgeyle yapar; START ve RESET_STATS burayı kullanır. Kayıt dizisi dosya düzeyinde static'tir; her olayda malloc yapılmaz.

RingBuffer_Alloc: sıradaki slotu seçmek

KAYNAK / Src/app_ring_buffer.c / satır 28-49

```

28 uint16_t RingBuffer_Alloc(uint16_t event_id, uint8_t scenario_id,
29                          uint32_t t0, uint32_t t0_abs_ms)
30 {
31     UBaseType_t saved = taskENTER_CRITICAL_FROM_ISR();
32
33     uint32_t idx = s_total_allocated % APP_RING_BUFFER_CAPACITY;
34     if (s_total_allocated >= APP_RING_BUFFER_CAPACITY) {
35         s_counters.ring_overflow_count++;
36     }
37     s_total_allocated++;
38
39     EventRecord_t *r = &s_records[idx];
40     memset(r, 0, sizeof(*r));
41     r->event_id = event_id;
42     r->scenario_id = scenario_id;
43     r->t0 = t0;
44     r->t0_abs_ms = t0_abs_ms;
45     r->status = (uint8_t)REC_STATUS_PENDING;
46     r->valid = true;
47
48     taskEXIT_CRITICAL_FROM_ISR(saved);
49     return (uint16_t)idx;

```

s_total_allocated, elde tutulan kayıt sayısı değil, bu senaryoda şimdiye kadar ayrılmış toplam sayıdır. % 256, fiziksel dizi indeksini 0..255 aralığına döndürür. İlk 256 olay sırasıyla tüm slotları doldurur. 257'nci olay tekrar indeks 0'a gelir ve en eski kaydın üzerine yazılır. Bu aşamadan itibaren her yeni ayırmada ring_overflow_count artar.

Seçilen slot önce sıfırlanır. Kimlik, senaryo, t0 ve kronoloji yazılır. status PENDING, valid true yapılır. t1..t4 henüz yoktur; ts_mask sıfır kalır. Fonksiyon indeks döndürür ve bu indeks ButtonEvent_t üzerinden diğer görevlere taşınır.

Kritik bölgenin ISR sürümü önceki kesme maskeleme durumunu saved içine alır; çıkarken geri yükler. ISR ve görev aynı diziye güncellediği için ayırma işleminin ortasında başka bir ilgili güncellenmenin girmesi engellenir. Bu koruma bütün sistem kesmelerinin koşulsuz kapalı olduğu anlamına gelmez; kullanılan FreeRTOS portunun maskeleme eşliğine bağlıdır.

Neden hem ring_index hem event_id?

KAYNAK / Src/app_ring_buffer.c / satır 52-65

```

52 /* Kritik bölge ICINDEN cagrilir. Slot artik bu olaya ait degilse NULL. */
53 static EventRecord_t *slot_for(uint16_t ring_index, uint16_t event_id)
54 {
55     if (ring_index >= APP_RING_BUFFER_CAPACITY) {
56         s_counters.id_mismatch_count++;
57         return NULL;
58     }
59     EventRecord_t *r = &s_records[ring_index];
60     if (!r->valid || r->event_id != event_id) {
61         s_counters.id_mismatch_count++;
62         return NULL;
63     }
64     return r;
65 }

```

İndeks, "hangi kutu?" sorusunu cevaplar. event_id, "bu kutuda hâlâ benim olayım mı var?" sorusunu cevaplar. Kuyrukta uzun süre bekleyen eski bir öge işlenene kadar ring dönmüş olabilir. Örneğin indeks 0 eskiden olay 1'iken artık olay 257'ye aitse, olay 1'in gecikmiş tamamlanması yeni kaydı değiştirmemelidir.

slot_for önce indeks sınırına bakar; sonra valid ve event_id eşleşmesini denetler. Uyuşmazlıkta id_mismatch_count artar ve NULL döner. Çağırın yazma fonksiyonu NULL görürse kaydı değiştirmez. Bu koruma kimliklerin ayırt edilebilir

olduğu olay ömrü içindir; 16 bit event_id sınırsız benzersiz kimlik değildir. slot_for kendi başına kritik bölge açmaz; onu çağıran fonksiyon zaten koruma içindedir.

Damgaları ekleyen ve kaydı kapatan fonksiyonlar

Fonksiyon	Çağıran yer	Kayda / sayaca etkisi
RingBuffer_SetT1T2	ButtonTask, TX Put sonrası	t1/t2 yazılır; iki maske biti eklenir
RingBuffer_SetT3	send_item, gönderim girişimi sonrası	t3 yazılır; T3 biti eklenir
RingBuffer_CloseSuccess	send_item, TX_OK	t4 ve T4 biti; OK/LATE; success_count
RingBuffer_CloseDropIsr	Button ISR, ButtonQueue dolu	DROP; btn_queue_drop_count
RingBuffer_CloseDropTx	ButtonTask, TX Put başarısız	DROP; tx_queue_drop_count
RingBuffer_CloseTxErr	send_item, başlatma/DMA hatası	TX_ERR; tx_start_err_count
RingBuffer_CloseTimeout	send_item, bildirim bekleme aşımı	TIMEOUT; tx_timeout_count

CloseDropTx, CloseTxErr ve CloseTimeout, aynı işi üç kere yazmak yerine `close_with(index, id, status, &counter)` yardımcısını kullanır. `close_with` kritik bölge açar, slotu doğrular, durumu yazar ve adresi verilen sayacı artırır. CloseDropIsr aynı fikri kesme bağlamına uygun kritik bölgeyle gerçekleştirir.

KAYNAK / Src/app_ring_buffer.c / satır 90-107

```

90 void RingBuffer_CloseSuccess(uint16_t ring_index, uint16_t event_id, uint32_t t4)
91 {
92     taskENTER_CRITICAL();
93     EventRecord_t *r = slot_for(ring_index, event_id);
94     if (r != NULL) {
95         r->t4 = t4;
96         r->ts_mask |= (uint8_t)REC_TS_T4;
97         /* Mikrosaniye cinsinden karşılaştırma: ms'ye kesme 20.9 ms'yi OK sayardı. */
98         if (Timestamp_DeltaToUs(r->t0, t4) <= APP_DEADLINE_US) {
99             r->status = (uint8_t)REC_STATUS_OK;
100         } else {
101             r->status = (uint8_t)REC_STATUS_LATE;
102             s_counters.late_count++;
103         }
104         s_counters.success_count++;
105     }
106     taskEXIT_CRITICAL();
107 }
```

CloseSuccess, t4-t0 farkını önce mikrosaniyeye çevirir ve 20000 ile karşılaştırır. 20000 us dahil OK; dönüştürülmüş değer 20000'den büyükse LATE. late_count yalnızca LATE için artar, success_count her iki durumda da artar. Burada "success" zamanında olmayı değil, aktarımın tamamlanmasını ifade eder. Dönüşüm tamsayı us olduğu için sınıflama mikrosaniye çözünürlüğündedir.

Sayaç okuma ve küçük artırıcılar

RingBuffer_GetCounters, sayaç yapısını kritik bölge içinde yerel bir kopyaya alır ve onu döndürür. Biçimlendirme daha sonra bu tutarlı kopya üzerinden yapılır; snprintf boyunca kritik bölgede kalınmaz.

`inc_counter(uint32_t *counter)` ortak artırma yardımcısıdır. RingBuffer_IncCmdCrcErr komut CRC uyumsuzluğunu, RingBuffer_IncTelDrop kuyruğa konamayan TEL'i, RingBuffer_IncTelTxFail aktarımı başarısız TEL'i bu yardımcı üzerinden sayar. Bunlar belirli bir butonun zaman damgasını değiştirmez.

DumpNext: fiziksel dizi sırası ile olay sırası

KAYNAK / Src/app_ring_buffer.c / satır 167-181

```
167 bool RingBuffer_DumpNext(uint32_t *cursor, EventRecord_t *out)
168 {
169     bool found = false;
170     taskENTER_CRITICAL();
171     bool wrapped = s_total_allocated > APP_RING_BUFFER_CAPACITY;
172     uint32_t count = wrapped ? APP_RING_BUFFER_CAPACITY : s_total_allocated;
173     if (*cursor < count) {
174         uint32_t start = wrapped ? (s_total_allocated % APP_RING_BUFFER_CAPACITY) : 0u;
175         *out = s_records[(start + *cursor) % APP_RING_BUFFER_CAPACITY];
176         (*cursor)++;
177         found = out->valid;
178     }
179     taskEXIT_CRITICAL();
180     return found;
181 }
```

cursor, çağıranın kaç kayıt okuduğudur; ilk değeri sıfırdır. Toplam 256'yı aşmamışsa en eski kayıt indeks 0'dadır. Aşmışsa en eski mevcut kayıt `s_total_allocated % 256` indeksindedir. `(start + cursor) % 256` her adımda kronolojik sıradaki slotu seçer. Kayıt out'a kopyalanır, cursor artırılır ve valid bilgisi döndürülür.

Örneğin 300 olay ayrılmışsa toplam 44 eski kayıt üzerine yazılmıştır. En eski mevcut olay 45, son olay 300'dür. Dump 0..255 fiziksel indekslerini körlemesine gezmez; olay 45'in bulunduğu yerden başlayarak sarar. Bu nedenle bilgisayara eski-yeni sırasıyla veri gider.

BÖLÜM 16 / 22

16 Protokol: kaydı 64 bayta dönüştürmek

app_protocol.c iki yönü bir arada tanımlar: dışarıya çerçeve oluşturmak ve içeriden gelen komut satırını çözmek. Komut çözme yolunu gördük. Şimdi tüm göndericilerin paylaştığı çerçeve oluşturma yoluna bakalım. Bu fonksiyonlar UART'a göndermez; yalnızca verilen out dizisini doldurur.

Sabit çerçevenin düzeni

0..61 ASCII + boşluk 62 bayt	62 CRC-8 1 bayt	63 LF 1 bayt
---	------------------------------	---------------------------

Her çıkış çerçevesi toplam 64 bayt. CRC ilk 62 baytı, boşluklar dahil, kapsar.

Konum	İçerik	Nasıl hazırlanır?
0..61	En fazla 62 bayt ASCII içerik ve boşluk dolgusu	snprintf ile içerik; kalan yerlere boşluk
62	Ham CRC-8 baytı	Dolgu dahil ilk 62 bayt üzerinden
63	LF, yani 0x0A	Sabit satır sonu

Her mesajın toplam boyutunun aynı olması, UART'a hep APP_FRAME_SIZE verilmesini ve farklı mesajların hat üzerindeki boyutunun aynı kalmasını sağlar. Çerçevenin sonunda C string NUL'u gönderilmez. CRC ham bayt olduğu için çerçevenin tamamı okunabilir metin değildir; CRC'nin değeri LF'ye de eşit olabilir. Çıkış protokolünü yalnızca newline ile metin satırı gibi düşünmek doğru olmaz.

KAYNAK / Src/app_protocol.c / satır 23-29

```

23 static void pack_payload(uint8_t out[APP_FRAME_SIZE], const char *content, size_t len)
24 {
25     memcpy(out, content, len);
26     memset(out + len, ' ', APP_FRAME_PAYLOAD_SIZE - len);
27     out[APP_FRAME_CRC_INDEX] = Protocol_Crc8(out, APP_FRAME_PAYLOAD_SIZE);
28     out[APP_FRAME_LF_INDEX] = APP_FRAME_LF_BYTE;
29 }

```

pack_payload önce içeriği kopyalar, kalan payload alanını boşlukla doldurur, sonra CRC'yi hesaplar ve en sona LF koyar. **Dolgu CRC'den önce yapılır**, çünkü alıcıya giden ilk 62 baytın tamamı doğrulanmalıdır.

Protocol_Crc8 ne hesaplıyor?

KAYNAK / Src/app_protocol.c / satır 8-19

```
8 uint8_t Protocol_Crc8(const uint8_t *data, size_t len)
9 {
10     uint8_t crc = APP_CRC8_INIT;
11     for (size_t i = 0; i < len; i++) {
12         crc ^= data[i];
13         for (int b = 0; b < 8; b++) {
14             crc = (crc & 0x80u) ? (uint8_t)((crc << 1) ^ APP_CRC8_POLY)
15                               : (uint8_t)(crc << 1);
16         }
17     }
18     return crc;
19 }
```

crc başlangıçta 0'dır. Her bayt crc ile XOR'lanır. Ardından sekiz bit adımı yapılır: en yüksek bit set ise sola kaydırılmış değere 0x07 XOR'lanır; değilse yalnızca sola kaydırılır. uint8_t dönüşümü sonucu sekiz bitte tutar. Fonksiyon aynı bayt dizisinden aynı kontrol baytını üretir. Kullanıldığı iki yer, 62 baytlık çıkış payload'ı ve CRC alanı hariç giriş komut içeriğidir.

Burada CRC'nin görevi iletim sırasında bozulmuş içeriği ayırt etmeye yardımcı olmaktır. Uygulama bunu bir komut yetkilendirme mekanizması olarak kullanmaz. Bu kodu anlamak için polinom matematiğinin bütün teorisine ihtiyaç yok; aynı kuralların iki tarafta aynı baytlara uygulanması gerekir.

finish_frame: sığma kararı neden snprintf sonucundan?

snprintf döndürdüğü n ile, yeterli alan olsaydı kaç karakter yazacağını bildirir. Çerçeve kurucuları content[64] kullansa da payload sınırı 62'dir. finish_frame n'nin negatif olmadığını ve 62'yi aşmadığını denetler. Uygunsa pack_payload çağırır ve true döner.

Sığmıyorsa yarım kalmış bir TEL/REC göndermek yerine ERR, ENCODE_OVERFLOW, <tip>, <n> içeriği hazırlanır, paketlenir ve false döner. Hata metni de sığmazsa içerik uzunluğu sıfır seçilir. Uygulamadaki Build çağrılarının çoğu dönüşü (void) ile bırakır; yine de out içinde bu hata çerçevesi bulunduğundan gönderim akışı devam eder. Bu kodda encode hatası ayrıca bir buton TX_ERR durumuna çevrilmez.

Her Build fonksiyonunun sözleşmesi

Fonksiyon	ASCII içerik alanları	Kim kullanıyor?
Protocol_BuildBOOT	BOOT, sysclk_hz, dwt_ok, fw	UartTxTask başlangıcı
Protocol_BuildBTN	BTN, event_id, scenario, t0_abs_ms, d1_us, PRESSED	ButtonTask
Protocol_BuildTEL	TEL, seq, scenario, t_abs_ms, period, load_iter, load_us, drop, tx_fail, ring_ovf	TelemetryTask
Protocol_BuildREC	REC, event_id, scenario, t0_abs_ms, d1, d2, d3, d4, status	dump_send_all
Protocol_BuildSTAT	STAT, scenario, success, drop, tx_err, timeout, ring_ovf, cmd_crc_err, tel_drop, tel_tx_fail, id_mismatch	dump_send_all
Protocol_BuildDumpEnd	DUMP_END	dump_send_all en son

BOOT, BTN, TEL ve STAT alanlarını `snprintf` ile kendi yerel content dizilerine yazar ve `finish_frame`'e verir. `%1u` için `unsigned long`, `%u` için `unsigned` dönüşümleri biçim dizesinin beklediği türü sağlar. `DUMP_END` sabit bir içeriktir; `sizeof(content) - 1` son NUL karakterini gönderilecek uzunluğa katmaz.

TEL'deki `drop = btn_queue_drop_count + tx_queue_drop_count`; `tx_fail = tx_start_err_count + tx_timeout_count`. Bunlar buton olayının kayıp/hata toplamıdır. TEL'in kendisinin düşmesi `tel_drop_count`'ta ayrıca tutulur ve STAT'ta raporlanır. STAT success değeri OK+LATE toplamıdır; `late_count` ayrı tutulsa da bu STAT biçiminde ayrıca gönderilmez. LATE bilgisi REC durumlarında bulunur.

REC süreleri aşama süresi değildir

`Protocol_BuildREC` her damga için `fmt_delta` çağırır. Maske biti yoksa - döner. Varsa

`Timestamp_DeltaToUs(rec->t0, t)` sonucu metne yazılır. Dolayısıyla **d1,d2,d3,d4'ün hepsi t0'a göredir**. Ham cycle değerleri gönderilmez.

Örneğin REC, 7, 2, 1500, 40, 120, 1800, 2500, OK içeriği, olay 7'nin deneyin 1500'üncü milisaniyesinde başladığını ve t0'a göre 40/120/1800/2500 us noktalarına ulaştığını söyler. $A1=40$, $A2=120-40=80$, $A3=1800-120=1680$, $A4=2500-1800=700$ us; $R=2500$ us. Bu sayılar öğretici örnektir, kart ölçümü değildir.

Her delta ayrı ayrı tamsayı us'ye çevrildiği için d alanlarını birbirinden çıkararak bulunan aşamalar ile ham cycle farkını ayrı dönüştürmek arasında küçük yuvarlama/kesme farkı olabilir. `status_to_str`, sayısal kayıt durumunu OK/LATE/DROP/TX_ERR/TIMEOUT metnine çevirir; diğer değerler PENDING olur.

`Protocol_BuildREC` içindeki `b1,b2,b3,b4` dizileri ayrı tutulur. `snprintf`'e verilen dört süre aynı geçici karakter dizisine yazılıyorsa, alanlar birbirinin içeriğini paylaşırdı. Her süreye ayrı yer ayırmak, fonksiyon argümanlarının değerlendirme sırasına bağlı olmadan dört metni birlikte kullanmayı sağlar.

BÖLÜM 17 / 22

17 Deney sonu: STOP'tan DUMP_END'e

Bir canlı BTN çerçevesi gönderilirken onun t4 değeri henüz bilinemez. Bu yüzden ayrıntılı sonuçların sonradan aktarılması yalnızca trafiği azaltma tercihi değildir; ölçümün zaman sırasıyla da uyumludur. Deney sırasında olayın parçaları RAM'de birleşir; DUMP bu tamamlanmış görünümü dışarı taşır.

KAYNAK / Src/app_tasks.c / satır 90-105

```

90 static void dump_send_all(void)
91 {
92     uint32_t      cursor = 0u;
93     EventRecord_t rec;
94     uint8_t       frame[APP_FRAME_SIZE];
95
96     while (RingBuffer_DumpNext(&cursor, &rec)) {
97         (void)Protocol_BuildREC(frame, &rec);
98         send_meta(frame);
99     }
100     RingCounters_t counters = RingBuffer_GetCounters();
101     (void)Protocol_BuildSTAT(frame, g_cfg.scenario_id, &counters);
102     send_meta(frame);
103     (void)Protocol_BuildDumpEnd(frame);
104     send_meta(frame);
105 }

```

dump_send_all cursor=0 ile başlar. RingBuffer_DumpNext bir kayıt kopyası döndürdüğüce Protocol_BuildREC ile 64 baytlık frame oluşturulur ve send_meta ile doğrudan gönderilir. Kayıt kalmayınca sayaçların bir kopyası alınır; STAT oluşturulur. En son DUMP_END gönderilir.

Bu döngü ring kayıtlarını silmez. DUMP yeniden istenirse mevcut kayıtlar yeniden en baştan dolaşılır. Hiç olay yoksa REC döngüsü hiç çalışmaz; yine STAT ve DUMP_END gönderilir. Metadata aktarımı başarısız olursa send_meta'nın tek tekrar denemesi uygulanır; burada sınırsız tekrar veya ayrı hata sayacı yoktur.

Tek görev olmasının sonucu

Dump gönderilirken UartTxTask aynı anda ana döngüsünde yeni komut ayrıştıramaz. RX kesmesi çalışıp CmdQueue'ya yeni satırlar koyabilir; fakat işleme sırası dump_send_all bittikten sonra gelir. Bu, komut alma ile komut uygulamanın neden ayrı aşamalar olduğunun ikinci örneğidir.

Kayıtlar deney sırasında normal TX trafiğine eklenmediği için ölçümün her olayına fazladan REC çerçevesi bindirilmez. Buna rağmen kayıt ayırma, sayaç güncelleme ve kısa kritik bölgeler yazılım maliyeti taşır. "Sonradan dump" ifadesi ölçüm kodunun maliyetinin tamamen sıfır olduğu anlamına gelmez; ayrıntılı UART raporlaması deney sonuna bırakılmıştır.

Başarısız olay da anlatılabilir

Durum	Var olabilecek damgalar	REC'de görülen anlam
ButtonQueue dolu	Yalnızca t0	d1,d2,d3,d4 ölçülmemiş; DROP
TX kuyruğuna konamadı	t0,t1,t2	d3,d4 yok; DROP
UART başlatma/DMA hatası	t0,t1,t2,t3	d4 yok; TX_ERR
TX tamamlanma bildirimi yetişmedi	t0,t1,t2,t3	d4 yok; TIMEOUT
Aktarım tamamlandı	t0..t4	OK veya LATE
Henüz kapanmamış kayıt	O ana kadar alınan damgalar	PENDING

Bir kayıt 20 ms'yi aştı diye DROP olmaz. DROP, kuyruğa teslim edilememe yoludur. LATE, aktarım tamamlandığı hâlde ölçülen R'nin sınırı aşmasıdır. TIMEOUT, TX tamamlanma bekleyişinin ayrı sınırıdır. Bu adları birbirinin yerine kullanmamak ölçümü doğru okumayı sağlar.

BÖLÜM 18 / 22

18 Kodda mevcut üç TX modu

Bu bölüm alternatif tasarım önerisi yapmıyor. Aynı kaynakta zaten yazılı üç derleme dalını açıklıyor. Seçim `app_config.h` makrolarıyla yapılır. Çalışırken gelen CFG yalnızca periyot, iterasyon ve senaryo etiketini değiştirir; kuyruk sayısını değiştirmez.

Özellik	ORIGINAL	LARGE_FIFO (mevcut varsayılan)	DUAL_QUEUE
TX yapısı	Ortak FIFO	Ortak FIFO	TEL FIFO + ayrı BTN FIFO
TxQueue kapasitesi	16	256	16, TEL için
BtnTxQueue	Yok	Yok	8, BTN için
BTN Put beklemesi	En çok 200 tick	En çok 200 tick	0
TEL Put beklemesi	En çok 200 tick	En çok 200 tick	0
Gönderici boşken	TX Get, en çok 20 tick	TX Get, en çok 20 tick	Thread flag ile süresiz bekleme
Sıradaki mesajı alma	Ortak FIFO başı	Ortak FIFO başı	Önce BTN dene, yoksa TEL dene
Görev öncelikleri	Low / Normal / AboveNormal	Aynı	Aynı

ORIGINAL ve LARGE_FIFO arasında görev algoritması değişmez; kapasite değişir. Kapasite, kaç ögenin RAM'de bekleyebileceğidir; UART'ın bit hızını veya görevin CPU önceliğini değiştirmez. Ayrıca ring buffer kapasitesi bu modlardan bağımsız olarak 256'dır. TX kuyruğu 256 ile ring 256'nın aynı sayıyı taşıması aynı veri yapısı oldukları anlamına gelmez.

DUAL_QUEUE'daki seçme ve uyanma

KAYNAK / Src/app_tasks.c / satır 194-203

```

194 #ifdef APP_TX_MODE_DUAL_QUEUE
195     /* Task ONCELIKLERI AYNI. Gonderime baslamamis mesajlarda BTN > TEL.
196     * Aktif DMA aktarimi yarida kesilmez. */
197     if (osMessageQueueGet(BtnTxQueueHandle, &item, NULL, 0u) == osOK ||
198         osMessageQueueGet(TxQueueHandle, &item, NULL, 0u) == osOK) {
199         send_item(&item);
200         continue;
201     }
202     /* UartTxTask iki FIFO da bosken uyur; yeni mesaj/komut ile uyanir. */
203     (void)osThreadFlagsWait(APP_TX_WORK_READY_FLAG, osFlagsWaitAny, osWaitForever);

```

|| kısa devreli çalışır. Önce BtnTxQueue'dan Get denenir; başarı varsa sağdaki TxQueue Get yapılmaz. BTN yoksa TEL kuyruğuna bakılır. Başarılı alımdan sonra send_item çağrılır ve continue ile döngünün başına dönülür; komutlar yeniden kontrol edilir.

İki kuyruk da boşsa UartTxTask, APP_TX_WORK_READY_FLAG bekler. Bu bit hem yeni BTN/TEL hem de yeni komut için uyandırmadır. Bir flag kaç mesaj olduğunu saymaz; birden çok Set tek bit hâlinde birleşebilir. Asıl iş adedi kuyruklarda durur. Uyanan görev yeniden kuyruklara baktığı için her mesaj için ayrı bildirim sayısı gerekmez.

Başarılı Put'tan sonra flag set edilir; önce veri, sonra bildirim. Alıcı kuyruğu kontrol ettikten hemen sonra üretici flag set etmiş olsa da bit bekleme anına kadar tutulabilir. Varsayılan Wait davranışı karşılanan thread flag'i temizler; RunFlags'teki osFlagsNoClear ile bu yüzden aynı kullanım değildir.

Başlamış DMA yarıda kesilmez. BTN, gönderici yeni öge seçtiğinde ayrı kuyrukta bekliyorsa TEL'den önce seçilir. Bu cümle "BTN her koşulda derhal telden çıkar" anlamına gelmez: devam eden çerçeve ve CPU zamanlaması hâlâ vardır. BTN kontrolüyle TEL alma arasına yeni BTN gelirse zaten alınmış öge geri bırakılmaz.

Kaynak yorumunda DUAL_QUEUE'nun TEL Put'una dair düşük öncelikli görevin çalışmaması şeklinde bir açıklama bulunuyor. Çağrının gerçek RTOS anlamı şudur: timeout=0 üreticiyi kuyrukta yer bekletmez; başarısızlığı hemen sayar. Bloklü bir Put ise CPU'yu bırakır. Bu rehber yorumdaki ifadeyi bir scheduler kuralı olarak tekrarlamaz.

BÖLÜM 19 / 22

19 Bir olayı baştan sona birlikte izleyelim

Bu bölümdeki sayılar yalnızca öğrenme örneğidir. Gerçek karttan alınmış ölçüm değildir. Mevcut ortak FIFO düzenini ve senaryo etiketi 2'yi düşünelim.

1. PC önce CFG ile periyot/yük/etiket gönderir. Satır RX kesmesinde birikir, CmdQueue'ya konur. UartTxTask CRC'yi ve alanları doğrular, g_cfg'yi günceller.
2. START aynı yolu izler. Ring kayıtları ve olay kimliği sıfırlanır, başlangıç ms'si alınır, g_running true ve RUN_BIT set olur. TelemetryTask ilk periyodunu beklemeye başlar.
3. Buton kenarı uygulama işleyicisine ulaşır. t0 alınır. Sessizlik, basılı seviye ve çalışma koşulları sağlanır. Olay 7 için bir ring slotu açılır ve ButtonEvent_t ButtonQueue'ya konur.
4. ButtonTask CPU'yu aldığı anda Get döner. t1, t0'dan 40 us sonradır. BTN oluşturulur. t2, t0'dan 120 us sonradır. TxQueue'ya BTN'nin kopyası konur; t1/t2 ring'e yazılır.
5. Kuyrukta önceden TEL mesajları olabilir. UartTxTask onları sırayla alır. BTN'nin sırası geldiğinde s_tx_buffer hazırlanır. t3, t0'dan 1800 us sonra alınır; DMA başlatılır.
6. UartTxTask semaphore'da blokludur. UART tamamlanma callback'i t4'ü t0'dan 2500 us sonra alır ve semaphore bırakır. Görev yeniden çalıştığı anda bu önceden alınmış t4'ü okur.
7. send_item ilgili ring slotuna t3'ü yazar ve CloseSuccess ile t4/OK sonucunu işler. success_count artar. BTN çerçevesi kendi gönderiminden sonraki bu sonucu içeremez.
8. PC STOP gönderir. Üretim kapanır; bekleyen mesajlar tüketilir. DUMP koşulu sağlandığında olayın REC kaydı 40,120,1800,2500 us alanlarıyla gönderilir. Ardından STAT ve DUMP_END gelir.

40 us	80 us	1680 us	700 us
A1: teslim	A2: hazırlama	A3: TX yolunda	A4: aktarım
d1 = 40	d2 = 120	d3 = 1800	d4 = R = 2500

Öğretici örnek, ölçüm değil. Aşamalar ayrı sürelerdir; d alanları t0'a göre birikimli sürelerdir.

Örnekte $R=2500$ us ve $A1+A2+A3+A4=40+80+1680+700=2500$ us. UartTxTask ancak 4000 us anında yeniden çalışabilmiş olsaydı da callback 2500 us'ta alınmışsa kayıt $R=2500$ us olurdu. Bitiş damgasının kesmede alınmasının etkisi tam olarak budur.

Aynı olay kimliği üç yerde nasıl geziyor?

Aşama	Nesne	Olayla bağı
Kabul	EventRecord_t	event_id=7 ile RAM kaydı açılır
ISR -> görev	ButtonEvent_t	event_id ve ring_index taşınır
Görev -> gönderici	TxItem_t	Aynı iki alan, hazır frame ile birlikte taşınır
Tamamlanma	slot_for(index,id)	İlgili slot hâlâ olay 7'ye mi ait doğrulanır
Bilgisayara kayıt	REC içeriği	event_id ve scenario_id dışarı çıkar; fiziksel ring_index gönderilmez

ring_index, MCU içindeki depolama adreslemesidir. event_id, okuyucunun olayı takip ettiği kimliktir. Bu ayrım veri yapılarının neden birbirinin birebir kopyası olmadığını açıklar.

BÖLÜM 20 / 22

20 Yalnızca bu projede kullanılan RTOS çağrıları

Bu tablo bir FreeRTOS ders kitabı değildir. Kaynakta gördüğümüz çağrıların bu uygulamadaki anlamını toplar. CMSIS-RTOS2 adları uygulamanın kullandığı arayüzdür; FreeRTOS bu projede alttaki çekirdektir.

Çağrı / seçenek	Buradaki anlamı
osKernelInitialize / osKernelStart	Nesne kurulumunu hazırlamak / scheduler'ı başlatmak
osThreadNew(entry,NULL,&attr)	entry görevini belirtilen öncelik ve bayt cinsinden yığınla oluşturmak
osMessageQueueNew(n,sizeof(T),&attr)	n adet T kopyası taşıyacak kuyruk oluşturmak
osMessageQueuePut(q,&x,0,timeout)	x'in kopyasını teslim etmek; doluya timeout kuralını uygulamak
osMessageQueueGet(q,&x,NULL,timeout)	Sıradaki öğeyi çıkarıp x'e kopyalamak; boşsa bekleme kuralını uygulamak
osMessageQueueGetCount(q)	O anda kuyrukta bekleyen öğe sayısı; görev elindeki öğeyi saymaz
osSemaphoreNew(1,0,&attr)	Başlangıçta boş, en fazla tek bildirim tutan semaphore
osSemaphoreAcquire(s,0)	Varsa jetonu tüket; yoksa beklemeden dön
osSemaphoreAcquire(s,100)	TX sonucu için en çok 100 tick bloklan
osSemaphoreRelease(s)	Callback'ten göndericiyi uyandıracak jetonu bırak
osEventFlagsNew / Set / Clear	RUN_BIT durum nesnesini oluştur / deneyi aç / çalışma iznini kaldır
osEventFlagsWait(...NoClear,Forever)	RUN_BIT'i bekle, karşılanınca otomatik silme
osKernelGetTickCount	Periyodik hedef tarihi hesaplamak için kernel sayacı
osDelayUntil(wake)	Mutlak hedef tick'e kadar bloklan
osDelay(1)	Bir tick görelî bekleme; geçişte veya defaultTask'ta CPU'yu bırakma
osThreadFlagsSet / Wait	DUAL_QUEUE'da tek göndericiye iş geldiğini bildirme / iş bekleme
taskENTER_CRITICAL / EXIT	Görev bağlamında ring kayıt ve sayaç güncellemesini koruma
taskENTER_CRITICAL_FROM_ISR / EXIT...	Aynı korumayı ISR bağlamında önceki maske durumunu saklayarak yapma

timeout=0, "hiç bekleme" demektir. osWaitForever, "koşul oluşana kadar bloklu kal" demektir. Bunlar CPU'yu meşgul eden boş döngüler değildir. osOK başarılı çağrı sonucudur; kaynak bazı yerlerde bütün diğer durumları aynı başarısızlık yoluna toplar.

Resmî API sözleşmesi de ISR'dan queue çağrılarında timeout'un 0 olmasını ister: [Arm CMSIS-RTOS2 Message Queue](#). Buradaki kullanımın gerekçesi budur; kütüphanenin queue iç algoritmasını açmıyoruz.

Thread flag'in uyandırma için kullanımı [Arm CMSIS-RTOS2 Thread Flags](#) sözleşmesiyle; ISR kritik bölgesinde önceki maske durumunun geri yüklenmesi [FreeRTOS ISR kritik bölge API'si](#) ile doğrulandı. Projeye özel davranış için esas alınan kaynak, projenin kendi kodudur.

Üç öncelik kavramını ayrı okuyun

Görev önceliğinde AboveNormal, Normal'den yüksektir. NVIC kesme önceliğinde ise küçük sayılar daha yüksek aciliyet demektir; bu projede ilgili EXTI/UART/DMA kesmelerinin sayısal önceliği 5, RTOS API eşiği de 5'tir. Queue Put'un mesaj önceliği argümanı üçüncü bir alandır; burada 0 kullanılır ve mevcut uyarlamada göz ardı edilir. Bu üç değeri aynı sıralama çizelgesinin üyeleri gibi karşılaştırmayın.

UART erişimi için uygulamada mutex oluşturulmaz; gönderimin tek sahibi zaten UartTxTask'tır. Ring içinse kısa kritik bölgeler vardır, çünkü hem ISR hem görev aynı kayıtları değiştirir. TxDoneSem UART sahipliğini paylaştıran bir kilit değil, tamamlanma bildirimidir. Kaynakta her senkronizasyon nesnesi farklı bir ihtiyaca karşılık gelir.

BÖLÜM 21 / 22

21 Dosyalar, bağımlılıklar ve fonksiyon bulma kılavuzu

Bir dosyanın başka dosyayı include etmesiyle bir fonksiyonun diğerini çağırması farklı ilişkilerdir. Başlık bağımlılığı, "bu tip/ad bilinmeli" demektir. Çalışma zamanı bağımlılığı, "bu işin yapılması için şu iş yapılmalı" demektir. Aşağıdaki harita ikinci ilişkiyi öne çıkarır.

Modül	Sağladığı iş	Bağlandığı uygulama modülleri
app_config.h	Ortak sabitler, mod seçimi	Başka modüller bunu tüketir
app_rtos.h / .c	Ortak veri tipleri, handles, durum, kurulum	timestamp, ring, button, cmd_rx, tasks
app_button.h / .c	Kenarı olay yapma	config, rtos, ring, timestamp
app_cmd_rx.h / .c	UART baytlarını satır yapma	hw, config, rtos
app_tasks.c	Üreticiler, tek gönderici, kontrol ve dump	rtos, protocol, ring, timestamp, button, hw
app_hal_callbacks.c	Donanım olayını uygulamaya bağlama	rtos, timestamp, button, cmd_rx, hw
app_ring_buffer.h / .c	Kayıtlar, sonuçlar, sayaçlar	config, timestamp ve kullanılan kritik bölge API'leri
app_protocol.h / .c	Mesaj üretimi ve komut çözme	config, ring veri tipleri, timestamp
app_timestamp.h / .c	Cycle damgası ve us dönüşümü	İşlemci sayaç tanımları ve SystemCoreClock
app_hw.h	Tek UART nesnesinin extern bildirimi	main.h ve config

İhtiyaçtan koda gitmek

Aklınızdaki soru	İzlenecek zincir
Başış neden kabul edilmedi?	Callback pin filtresi -> Button_HandleEXTI koşulları -> ButtonQueue Put sonucu
Yanıtın içeriği nereden geliyor?	ButtonTask_Run -> Protocol_BuildBTN -> finish_frame -> pack_payload
Gönderim bittiğini kim söylüyor?	HAL_UART_TxCpltCallback -> g_tx_done_cycles + TxDoneSem -> uart_tx_send
Geç kalan olay nerede sınıflanıyor?	send_item -> RingBuffer_CloseSuccess -> Timestamp_DeltaToUs
STOP neden anında işlenmeyebilir?	RX satırı -> CmdQueue -> UartTxTask'ın ana döngüsüne dönüşü -> kontrol
DUMP sırasında hangi veri gönderiliyor?	RingBuffer_DumpNext -> Protocol_BuildREC -> send_meta -> uart_tx_send
Yanlış komut nasıl eleniyor?	Protocol_ParseCommandLine -> CRC/biçim ayrımı -> handle_cmd_line

Uygulama fonksiyonlarının tam dizini

Aşağıdaki dizin kaynaklardan otomatik çıkarıldı. Fonksiyonların yalnızca bildirimlerini değil, gövdelerinin başladığı satırları gösterir. static yardımcılar ve header içindeki Timestamp_Now dahildir. HAL sürücü ve RTOS çekirdek fonksiyonları dizine alınmadı; uygulamanın yazdığı HAL isimli callback'ler dahildir.

Fonksiyon	Kaynak başlangıcı	Açıklama bölümü
Button_Init	Src/app_button.c:12	10
Button_ResetForScenario	Src/app_button.c:19	10
Button_HandleEXTI	Src/app_button.c:24	10
rearm	Src/app_cmd_rx.c:11	07
CmdRx_Init	Src/app_cmd_rx.c:17	07
CmdRx_HandleByteReceived	Src/app_cmd_rx.c:24	07
CmdRx_OnError	Src/app_cmd_rx.c:54	07
HAL_UART_TxCpltCallback	Src/app_hal_callbacks.c:17	14
HAL_UART_ErrorCallback	Src/app_hal_callbacks.c:29	14
HAL_UART_RxCpltCallback	Src/app_hal_callbacks.c:48	14
HAL_GPIO_EXTI_Callback	Src/app_hal_callbacks.c:55	14
Protocol_Crc8	Src/app_protocol.c:8	16
pack_payload	Src/app_protocol.c:23	16
finish_frame	Src/app_protocol.c:35	16
status_to_str	Src/app_protocol.c:51	16
fmt_delta	Src/app_protocol.c:64	16
Protocol_BuildTEL	Src/app_protocol.c:75	16
Protocol_BuildBTN	Src/app_protocol.c:91	16
Protocol_BuildBOOT	Src/app_protocol.c:102	16
Protocol_BuildREC	Src/app_protocol.c:111	16
Protocol_BuildSTAT	Src/app_protocol.c:126	16
Protocol_BuildDumpEnd	Src/app_protocol.c:145	16
hex_nibble	Src/app_protocol.c:153	08
parse_u32	Src/app_protocol.c:161	08
next_token	Src/app_protocol.c:175	08

Fonksiyon	Kaynak başlangıcı	Açıklama bölümü
Protocol_ParseCommandLine	Src/app_protocol.c:189	08
RingBuffer_Init	Src/app_ring_buffer.c:12	15
RingBuffer_ResetForScenario	Src/app_ring_buffer.c:19	15
RingBuffer_Alloc	Src/app_ring_buffer.c:28	15
slot_for	Src/app_ring_buffer.c:53	15
RingBuffer_SetT1T2	Src/app_ring_buffer.c:67	15
RingBuffer_SetT3	Src/app_ring_buffer.c:79	15
RingBuffer_CloseSuccess	Src/app_ring_buffer.c:90	15
RingBuffer_CloseDropIsr	Src/app_ring_buffer.c:109	15
close_with	Src/app_ring_buffer.c:120	15
RingBuffer_CloseDropTx	Src/app_ring_buffer.c:132	15
RingBuffer_CloseTxErr	Src/app_ring_buffer.c:137	15
RingBuffer_CloseTimeout	Src/app_ring_buffer.c:142	15
RingBuffer_GetCounters	Src/app_ring_buffer.c:147	15
inc_counter	Src/app_ring_buffer.c:156	15
RingBuffer_IncCmdCrcErr	Src/app_ring_buffer.c:163	15
RingBuffer_IncTelDrop	Src/app_ring_buffer.c:164	15
RingBuffer_IncTelTxFail	Src/app_ring_buffer.c:165	15
RingBuffer_DumpNext	Src/app_ring_buffer.c:167	15
App_RunElapsedMs	Src/app_rtos.c:30	06
require	Src/app_rtos.c:35	06
App_RTOS_Init	Src/app_rtos.c:43	06
uart_tx_send	Src/app_tasks.c:27	14
send_item	Src/app_tasks.c:56	13
send_meta	Src/app_tasks.c:80	13
dump_send_all	Src/app_tasks.c:90	17
control_handle_command	Src/app_tasks.c:107	09
handle_cmd_line	Src/app_tasks.c:164	08

Fonksiyon	Kaynak başlangıcı	Açıklama bölümü
UartTxTask_Run	Src/app_tasks.c:179	13
ButtonTask_Run	Src/app_tasks.c:216	11
TelemetryTask_Run	Src/app_tasks.c:262	12
Timestamp_Init	Src/app_timestamp.c:5	05
Timestamp_DeltaToUs	Src/app_timestamp.c:20	05
Timestamp_Now	Inc/app_timestamp.h:19	05

İkinci okumada kullanacağınız kısa rota

1. Bir akışı seçin: açılış, START, tek basış, telemetri veya DUMP. Aynı anda tüm dosyaları ezberlemeye çalışmayın.
2. Her fonksiyonda önce "kim çağırıyor, hangi bağlamda?" sorusunu cevaplayın. ISR ile görev arasındaki sınırı işaretleyin.
3. Girişte alınan veri ile çıkışta verilen veriyi bulun. Queue'ya kopyalanan paket ve ring'e yazılan alanları birbirinden ayırın.
4. Bir bekleme gördüğünüzde nedenini okuyun: olay mı, periyot mu, boş yer mi, aktarım sonucu mu? Beklerken hangi görevlerin çalışabileceğini düşünün.
5. Zaman damgası gördüğünüzde alma anı ile kayda yazma anını ayırın. REC'deki alanların hepsinin t0'a göre olduğunu hatırlayın.

Bu sırayla, bir BTN'nin neden önce bir olay, sonra bir çerçeve, en son bir ölçüm kaydı olarak görüldüğü anlaşılır: her temsil, yolculuğun o aşamasının ihtiyaç duyduğu bilgiyi taşır.

BÖLÜM 22 / 22

22 Kaynak kapsamı ve okuma notları

Bu rehberdeki satırlı kod kutuları kaynak dosyalardan doğrudan alınmıştır. Yorumlar da orijinal biçimleriyle korunur; güncel davranışla uyuşmayan yorumlar açıklama metninde ayrıştırılır. HTML'nin kaynak bağlantıları, belge proje klasöründeki yerinde duruyorsa ilgili dosyayı açar. Belge başka yere kopyalansa da açıklamalar ve içindeki kod alıntıları çevrimdışı okunabilir.

İncelenen alan	Rehberdeki kapsam
Tüm Core/Src/app_*.c	Uygulama gövdeleri, static yardımcıları ve callback'ler
Tüm Core/Inc/app_*.h	Sabitler, veri sözleşmeleri, bildirimler, inline zaman okuma
main.c	App_RTOS_Init bağlantısı, scheduler sırası, baud seçimi, defaultTask'ın yeri, Error_Handler sonucu
FreeRTOSConfig.h	Kullanılan tick, heap, preemption ve ISR API eşikleri
stm32f7xx_it.c	Uygulama callback'lerine ulaşan IRQ yönlendirme zinciri
freertos.c	Mevcut hook gövdelerinin boş olduğu; burada uygulama görev kurulumu bulunmadığı
.cproject ve Debug kuralları	Derleme modu için harici tanım bulunup bulunmadığı
README, SPEC, docs/code-notes	Amaç ve tasarım gerekçesi için yardımcı kaynak; sayısal gerçeklikte kod esas alındı

Çalıştırılabilir PC arayüzünün kaynak kodları bu klasörün interface dizininde bulunmuyor; yalnızca .exe dosyaları görülüyor. Bu nedenle arayüzün iç algoritması açıklanmadı ve eski belgelerdeki otomatik deneme/kaydetme davranışları firmware'in kendi yaptığı işler gibi sunulmadı. Protokol, firmware sınırına kadar anlatıldı.

Bu çalışma kodu değiştirmeden anlamaya yöneliktir. Bir tasarım değiştirme önerisi veya alternatif mimari listesi içermez. "Neden burada?" açıklamaları kaynakta görülen sıra ve veri bağımlılıklarına dayanır. Örnek sayılar öğretici olarak etiketlendi; depo belgelerindeki geçmiş testler bu çalışmada yeniden yapılmış gibi gösterilmedi.